ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И ТЕХНОЛОГИИ“



ПРЕДОТВРАТЯВАНЕ НА ИНЖЕКЦИОННИ АТАКИ ЧРЕЗ КОНСТРУИРАНЕ НА ЗАЯВКИ ПО ВРЕМЕ НА КОМПИЛАЦИЯ: АНАЛИЗ И ОЦЕНКА ПРИ ХЕТЕРОГЕННИ ХРАНИЛИЩА ЗА ДАННИ

ДИПЛОМНА РАБОТА

за придобиване на ОКС „Магистър“

Изготвил:

Алекс Цветанов, фак. № 371326005

Специалност: Киберсигурност и превенция на киберпрестъпления

Научен ръководител:

[TODO: звание, име на ръководителя]

София, 2027

УТВЪРДЕНО ДИПЛОМНО ЗАДАНИЕ

[TODO: тук се подвързва утвърденото и подписано дипломно задание]

ДЕКЛАРАЦИЯ ЗА АВТОРСТВО

[TODO: тук се подвързва подписаната декларация за авторство по образеца на ФКСТ]

РЕЗЮМЕ

Инжекционните атаки остават сред най-разпространените уязвимости в софтуера и са средство за редица документирани киберпрестъпления. Настоящата дипломна работа изследва дали една абстракция, конструираща заявките към хранилища за данни по време на компилация, може да елиминира цяло семейство от класове инжекции при хетерогенни хранилища, и къде точно спира тази гаранция. Като платформа се използва библиотека за обектно-релационно картографиране на C++23, разработена в предходна дипломна работа, която работи със седем хранилища: PostgreSQL, MySQL, SQLite, Cassandra, MongoDB, Neo4j и Redis.

Въз основа на определението на инжекцията като промяна на синтактичната структура на заявката е формулиран аргумент: когато имената на таблици и колони и формата на заявката са аргументи на шаблони, а стойностите достигат заявката само чрез свързани типизирани заместители, недоверен вход не може да промени структурата. Аргументът е проверен с три независими метода: 15 отрицателни компилационни теста и 3 контролни случая, с проверка на диагностиката при GCC 14.2 и Clang 18.1; диференциален тест, при който всеки приложим полезен товар от корпус от 66 публикувани полезни товара от 11 източника се подава към съзнателно уязвима заявка и към библиотеката; и анализ на изходния код на всеки конектор.

Анализът установи, че гаранцията се постига с четири различни механизма, и откри пропуск в нея: стойност, записана направо в заявката вместо чрез заместител, се приемаше, но не се свързваше, и заявката тихо връщаше грешни резултати. Пропускът е затворен с проверка по време на компилация във всички седем входни точки, при запазено успешно изпълнение на 314-те съществуващи и 23-те нови теста. При SQLite наивната заявка беше компрометирана от 4 от 22 приложими полезни товара, включително с извличане на съхранена идентификационна информация, а библиотеката от нито един. Тестът беше изпълнен и срещу работещи сървъри на PostgreSQL, MySQL, MongoDB, Redis и Cassandra в система за непрекъсната интеграция. Общо за шестте хранилища нито един от 92-та приложими полезни товара не е имал ефект през библиотеката. Наивният път е компрометиран от по 2 полезни товара при PostgreSQL, MySQL, MongoDB и Redis. При Cassandra той не е компрометиран от нито един и сравнението е неубедително. Neo4j не е измерен.

Приносите на работата са аргумент за предотвратяване на инжекциите чрез конструиране по време на компилация, разширен извън SQL; възпроизводим набор за оценка с корпус с посочени източници; и открит и затворен пропуск в гаранцията. Измерването при Neo4j остава бъдеща работа.

Ключови думи: инжекционни атаки, предотвратяване на киберпрестъпления, сигурност по проектиране, C++, програмиране по време на компилация, хранилища за данни.

ABSTRACT

Injection attacks remain among the most common software vulnerabilities and are the means of a number of documented cybercrimes. This thesis examines whether an abstraction that constructs queries to data stores at compile time can eliminate a whole family of injection classes across heterogeneous stores, and where exactly that guarantee stops. The platform is a C++23 object-relational mapping library, developed in an earlier thesis, that works with seven stores: PostgreSQL, MySQL, SQLite, Cassandra, MongoDB, Neo4j and Redis.

Building on the definition of injection as a change to the syntactic structure of a query, the thesis argues that when table names, column names and the shape of the query are template arguments, and values reach the query only through bound, typed placeholders, untrusted input cannot change that structure. The argument is tested by three independent methods: 15 negative compilation tests and 3 control cases, with diagnostic matching under GCC 14.2 and Clang 18.1; a differential test that sends every applicable payload from a corpus of 66 published payloads from 11 sources both to a deliberately vulnerable query and to the library; and an analysis of the source of every connector.

The analysis found that the guarantee is achieved by four different mechanisms, and found a gap in it: a value written directly into a query instead of through a placeholder was accepted but never bound, and the query silently returned wrong results. The gap is closed by a compile-time check at all seven entry points, with all 314 existing and 23 new tests passing. On SQLite the vulnerable query was compromised by 4 of 22 applicable payloads, including the theft of a stored credential, and the library by none. The test was then run against live servers of PostgreSQL, MySQL, MongoDB, Redis and Cassandra in a continuous integration system. Across the six stores, none of the 92 applicable payloads had an effect through the library. The naive path was compromised by 2 payloads each on PostgreSQL, MySQL, MongoDB and Redis. On Cassandra it was never compromised, so the comparison is inconclusive. Neo4j was not measured.

The contributions are an argument for preventing injection by compile-time construction, extended beyond SQL; a reproducible evaluation suite with a sourced corpus; and a gap in the guarantee, found and closed. Measurement on Neo4j remains future work.

Keywords: injection attacks, cybercrime prevention, secure by design, C++, compile-time programming, data stores.

СЪДЪРЖАНИЕ

УВОД **1**

I. Инжекциите като средство за киберпрестъпления **4**

1.1. Инжекцията в съвременния пейзаж на заплахите 4

1.2. Документирани случаи 5

1.3. Правна рамка 6

1.4. От наказание към превенция 10

II. Преглед на съществуващите решения **11**

2.1. Защити по слоеве 11

2.2. Академична работа по откриване и предотвратяване 13

2.3. Инжекции извън SQL 14

2.4. Типово безопасно конструиране на заявки 15

2.5. Сравнение на подходите 16

2.6. Пропускът 19

III. Теоретична основа **20**

3.1. Инжекцията като промяна на синтактичната структура 20

3.2. Три начина за отделяне на данните от кода 21

3.3. Езиците на изследваните хранилища 22

3.4. Времето на компилация като граница на доверие 23

IV. Модел на заплахите и метод **24**

4.1. Защитаван актив 24

4.2. Модел на нападателя 24

4.3. Извън обхвата 25

4.4. Изследователски въпрос и хипотеза 25

4.5. Критерии за успех 26

4.6. Метод на оценката 26

4.7. Какво методът не доказва 27

V. Реализация на гаранцията при седем хранилища **28**

5.1. Платформата 28
5.2. Откъде идва всяка част на заявката 28
5.3. Механизми при отделните хранилища 30
5.4. Проверка на границата 32
5.5. Известен остатъчен риск 33

VI. Казус: непривързаните литерални операнди 34
6.1. Откриване 34
6.2. Хипотеза и измерване 35
6.3. Анализ на причината 36
6.4. Избор на поправка 36
6.5. Проверка на поправката 37
6.6. Изводи от казуса 37

VII. Оценка 39
7.1. Среда и възпроизводимост 39
7.2. Корпус от полезни товари 40
7.3. Отрицателни компилационни тестове 41
7.4. Диференциален тест със SQLite 43
7.5. Тълкуване 45
7.6. Сървърните хранилища 45
7.7. Съпоставка с критериите за успех 49
7.8. Заплахи за валидността 50

ЗАКЛЮЧЕНИЕ 52

ИЗПОЛЗВАНИ ИЗТОЧНИЦИ 54

ПРИЛОЖЕНИЯ 60

УВОД

Актуалност

Инжекцията е една от най-старите и най-устойчивите уязвимости в софтуера. Тя продължава да заема видимо място в класациите на най-сериозните рискове за уеб приложенията [1] и е средство за документирани киберпрестъпления, обсъдени в глава I. Устойчивостта ѝ не се дължи на липса на знание. Защитата срещу класическата SQL инжекция е известна отдавна: стойностите трябва да се подават като параметри, а не да се вграждат в текста на заявката [2]. Уязвимостта оцелява, защото защитата зависи от дисциплината на всеки разработчик при всяка заявка.

Това е причината подходът към инжекциите да се измества от откриване и поправяне на отделни случаи към тяхното изключване като клас. Съвместно предупреждение на американските агенции CISA и FBI призовава производителите на софтуер да елиминират SQL инжекциите именно по този начин [3]. Възниква въпросът какви средства позволяват такова изключване и докъде то стига.

Изследователски въпрос

Настоящата работа изследва един такъв подход: конструиране на заявките по време на компилация. При него структурата на заявката, имената на таблиците и на колоните са фиксирани от типовата система, а стойностите могат да влязат само през типизирани заместители. Работата отговаря на въпроса:

> *Може ли една абстракция, конструираща заявките по време на компилация, да елиминира цяло семейство от класове инжекции при хетерогенни хранилища за данни, и къде точно спира тази гаранция?*

Въпросът е поставен за семейство от класове, а не само за SQL, защото съвременните приложения използват различни видове хранилища, всяко със собствен език и собствен клас инжекции. Втората част на въпроса е равностойна на първата. Гаранция, чиито граници не са известни, не може да бъде използвана безопасно.

Цел и задачи

Целта на работата е да провери, чрез измерване, а не само чрез разсъждение, доколко конструирането на заявки по време на компилация предотвратява инжекционните атаки при седем хетерогенни хранилища за данни, и да установи границите на тази защита. За постигане на целта са поставени следните задачи:

1. да се анализира инжекцията като средство за киберпрестъпления и правната рамка за противодействието ѝ;
2. да се прегледат съществуващите защити и да се установи незапълнената нужда;
3. да се формулира аргумент, свързващ формалното определение на инжекцията [4] с типовата система на C++;
4. да се определи модел на заплахите и критерии за успех, фиксирани преди измерванията;
5. да се анализира реализацията на гаранцията при всяко от седемте хранилища;
6. да се изгради възпроизводим набор за оценка: отрицателни компилационни тестове, корпус от публикувани полезни товари и диференциален тест;
7. да се оценят резултатите спрямо критериите и да се назоват границите на гаранцията.

Обхват

Обхватът е пътят на заявката от изграждането ѝ в приложението до предаването ѝ на драйвера на хранилището. Извън обхвата са операциите по схемата, инжекцията от втори ред, логическите грешки в приложенията, уязвимостите в самите драйвери и отказът от услуга чрез скъпи заявки. Тези граници са обосновани в точка 4.3.

Работата използва като платформа библиотека за обектно-релационно картографиране, разработена в предходна дипломна работа за ОКС „Бакалавър“ [5]. Предходната работа описва проектирането и реализацията на библиотеката и не разглежда въпроса за сигурността. Настоящата работа не претендира за нейните резултати. Тя изследва самостоятелен въпрос и формулира приноси, които се отнасят само към него и към промените в библиотеката, произтекли от изследването.

Структура на работата

Глава I разглежда инжекцията като средство за киберпрестъпления и правната рамка за противодействието ѝ. Глава II прави преглед на съществуващите защити и установява незапълнената нужда. Глава III въвежда формалното определение на инжекцията и аргумента за времето на компилация като граница на доверие. Глава IV определя модела на заплахите, критериите за успех и метода на оценката. Глава V анализира как гаранцията се постига при седемте хранилища. Глава VI проследява открит в хода на работата пропуск в гаранцията, от откриването до поправката. Глава VII представя и тълкува резултатите от оценката. Заключението обобщава приносите, ограниченията и насоките за бъдеща работа.

I. ИНЖЕКЦИИТЕ КАТО СРЕДСТВО ЗА КИБЕРПРЕСТЪПЛЕНИЯ

Уводът поставя въпроса дали един механизъм, действащ по време на компилация, може да изключи цял клас инжекции. Преди да се търси отговор, трябва да се покаже защо въпросът има значение извън инженерната практика. Настоящата глава разглежда инжекцията като средство за престъпления и като източник на отговорност за онези, които обработват чужди данни. Първо се очертава мястото на инжекциите в класациите на уязвимостите. След това се разглеждат три документирани случая и правната рамка: международна, европейска и българска. Накрая се разглежда преходът от наказване към превенция, който мотивира работата.

1.1. Инжекцията в съвременния пейзаж на заплахите

Инжекция в широк смисъл е ситуация, при която недоверен вход бъде изтълкуван от интерпретатор като част от команда, а не като данни. Формалното определение, което работата използва, се въвежда в глава III.

Действащото издание на класацията на *OWASP* за най-сериозните рискове при уеб приложенията е *OWASP Top 10:2025* [1]. В него инжекцията е на пето място (категория A05:2025), след като е слязла с две места от трето място в предишното издание. Категорията обединява 37 класа слабости по номенклатурата *CWE* и според документа е с най-голям брой регистрирани уязвимости (*CVE*) от всички категории [6].

Номенклатурата *CWE* (*Common Weakness Enumeration*) на *MITRE* описва видовете слабости, а не отделните уязвимости. Записът CWE-89 е озаглавен „Improper Neutralization of Special Elements used in an SQL Command (’SQL Injection’)“ [7]. Записът CWE-943 е озаглавен „Improper Neutralization of Special Elements in Data Query Logic“ и обхваща заявка към хранилище на данни, например база от данни, без да се ограничава до SQL [8]. Тази разлика в обхвата е съществена за работата. В класацията *2025 CWE Top 25 Most Dangerous Software Weaknesses* CWE-89 е на второ място, с едно място нагоре спрямо 2024 година [9]. Това е най-новият списък, наличен на страницата на проекта към 4 ноември 2026 година.

Тези източници не доказват, че инжекциите се използват за престъпления. Те показват, че класът е широко разпространен, устойчив във времето и добре описан, т.е. че защитата срещу него е въпрос на познати средства, а не на неизвестна заплаха.

1.2. Документирани случаи

За всеки случай има първичен или официален източник, който го свързва със SQL инжекция. Цифри са посочени само там, където са прочетени в такъв източник. Случаите илюстрират три правни положения: масово изнудване с откраднати данни, наказателно преследване на извършител и административна санкция на администратор на данни.

1.2.1. MOVEit Transfer и CL0P (2023)

Уязвимостта *CVE-2023-34362* засяга приложението *MOVEit Transfer* на *Progress Software*. Записът в Националната база данни за уязвимости на *NIST* я описва като SQL инжекция, която може да позволи на неудостоверен нападател достъп до базата от данни, класифицира я като CWE-89 и ѝ дава оценка 9.8 по *CVSS v3.1* [10]. Съвместното предупреждение *AA23-158A* на *FBI* и *CISA* от 7 юни 2023 година приписва кампанията на групата *CL0P*. Според него нападателите са използвали уязвимостта, за да инсталират уеб обвивката *LEMURLOOT* и да изтеглят данни от базите на приложението, а от 2021 година групата разчита главно на кражба на данни, а не на криптиране. Използването е започнало на 27 май 2023 година според публични източници, а на 2 юни 2023 година уязвимостта е добавена в каталога на активно използваните [11].

1.2.2. Heartland Payment Systems и Албърт Гонсалес (2009, 2010)

На 17 август 2009 година прокуратурата на щата Ню Джърси съобщава за обвинителен акт срещу трима души за пет корпоративни пробива, сред които *Heartland Payment Systems*, *7-Eleven* и *Hannaford*. Обвинителният акт описва кражба на над 130 милиона номера на кредитни и дебитни карти от тези три компании и определя атаките като „SQL-injection attack“. Съобщението напомня, че обвинителният акт е само обвинение [12]. На 26 март 2010 година Министерството на правосъдието на САЩ съобщава, че Албърт Гонсалес е осъден на 20 години и един ден лишаване от свобода по две обвинения в съзаговор във връзка с достъпа до платежните мрежи на тези три компании. Срокът се изтърпява едновременно с наложения предния ден в Бостън по друго дело [13]. Съобщението за присъдата не използва термина „SQL инжекция“. Приписването на метода се основава на обвинителния акт, а не на съдебно установяване.

1.2.3. TalkTalk и санкцията на ICO (2016)

На 5 октомври 2016 година британската Служба на комисаря по информацията (*Information Commissioner's Office*, *ICO*) налага на *TalkTalk* глоба от 400 000 британски лири, определена от службата като рекордна. Според нея атаката е осъществена от 15 до 21 октомври 2015 година чрез три уязвими уеб страници в наследена инфраструктура. Нападателят е получил достъп до личните данни на 156 959 клиенти, като при 15 656 от тях е имал достъп и до данни за банкови сметки [14]. Службата установява, че нападателят е използвал SQL инжекция, че техниката е „well understood", защити съществуват и компанията е трябвало да знае за риска. Имало е и две незабелязани предупреждения: успешна SQL инжекция на 17 юли 2015 година срещу същата уязвимост и втора атака между 2 и 3 септември 2015 година. Нарушението е квалифицирано като неспазване на седмия принцип на британския *Data Protection Act 1998*, а отделно наказателно разследване води Столичната полиция [14]. Санкцията е наложена по действалия тогава закон, а не по Общия регламент относно защитата на данните. Тя е за липса на мерки срещу добре позната атака, а не за самата атака.

Трите случая заемат три различни роли: обект на масово изнудване, извършител, подведен под наказателна отговорност, и администратор, санкциониран за липса на защита. Общото е, че слабостта е от клас, за който защитата е известна отдавна. *ICO* го казва за конкретния случай, а предупреждението на *CISA* и *FBI* (раздел 1.4), което говори за широко огласена кампания срещу приложение за управляван трансфер на файлове, засегнала хиляди организации, го казва за индустрията. То не назовава продукта [3]; връзката с *MOVEit* е извод на автора. Проблемът не е липсата на знание, а това, че знанието се прилага ръчно, на всяко място поотделно.

1.3. Правна рамка

Правното третиране на инжекцията се определя от резултата, а не от техниката. Нападател, който чрез инжекция получи достъп до данни, ги промени или спре работата на система, може да осъществи състава на престъпление по международни, европейски и национални актове. Администраторът на данните носи самостоятелно задължение да осигури защита. Прилагането на състав към конкретно деяние е въпрос на съдебна преценка, затова връзката между инжекцията и даден състав по-долу е формулирана като възможна правна квалификация. Съдебна практика не се разглежда.

1.3.1. Конвенцията от Будапеща

Конвенцията за престъпленията в кибернетичното пространство на Съвета на Европа (*ETS № 185*, Будапеща, 23 ноември 2001 година) е международен договор в областта на престъпленията в кибернетичното пространство [15]. Релевантни са три деяния от дял 1, посветен на престъпленията срещу поверителността, цялостността и наличността на компютърните данни и системи.[1] Член 2 (незаконен достъп, *illegal access*) задължава страните да установят като престъпление умишления достъп до компютърна система или част от нея без право. Страната може да изисква деянието да е извършено чрез нарушаване на мерки за сигурност или с нечестно намерение. Член 4 (намеса в данни, *data interference*) обхваща умишленото повреждане, изтриване, влошаване, променяне или скриване на данни без право. Член 5 (намеса в система, *system interference*) обхваща умишленото сериозно възпрепятстване на функционирането на система чрез въвеждане, предаване, повреждане, изтриване, влошаване, променяне или скриване на данни. За случаи като *Heartland* е релевантен и член 8 (компютърна измама, *computer-related fraud*) [15].

1.3.2. Директива 2013/40/ЕС

Директива 2013/40/ЕС относно атаките срещу информационните системи задължава държавите членки да предвидят като престъпления сходни деяния и замества Рамково решение 2005/222/ПВР [16]. Подредбата на членовете е различна от тази на Конвенцията: в Директивата член 3 урежда незаконния достъп до информационни системи, член 4 незаконната намеса в система, член 5 незаконната намеса в данни, а член 7 инструментите за извършване на престъпления. При Конвенцията системата е в член 5, а данните в член 4. Има и разлика по същество. Член 3 изисква деянието да е наказуемо, когато е извършено чрез нарушаване на мярка за сигурност, поне в случаите, които не се считат за маловажни, докато Конвенцията само дава на страните възможност да поставят това условие. Инжекция, която заобикаля удостоверяването, може да бъде разглеждана като преодоляване на мярка за сигурност, но това е въпрос на преценка по случая. Член 9 изисква максимален срок на лишаване от свобода най-малко две години за деянията по членове 3 до 7 и най-малко пет години за членове 4 и 5, когато са извършени в рамките на престъпна организация, причиняват сериозни вреди или са насочени срещу

[1] Заглавията на членовете на Конвенцията са дадени в превод на автора, защото официален български текст не беше намерен. Съдържанието е проверено по английския текст.

информационна система на критична инфраструктура [16].

1.3.3. Българският Наказателен кодекс

Компютърните престъпления са уредени в глава девета „а“ „Компютърни престъпления“, членове 319а до 319е на Наказателния кодекс [17]. Текстът е прочетен на lex.bg към 4 ноември 2026 година и не е сверяван с „Държавен вестник“. Посочват се само предметът на състава и размерът на лишаването от свобода. Глобите не се разглеждат.

Член 319а, ал. 1 наказва неправомерния достъп до информационна система или части от нея, в немаловажни случаи, с лишаване от свобода до шест години. За информация, представляваща държавна или друга защитена от закон тайна, и за информационна система, която е част от критична инфраструктура, ал. 4 предвижда от четири до осем години. Член 319б, ал. 1 наказва неправомерното добавяне, копиране, използване, променяне, пренасяне, изтриване, повреждане, влошаване, скриване или унищожаване на компютърни данни или спирането на достъпа до тях, в немаловажни случаи, с лишаване от свобода до шест години. При цел имотна облага ал. 3 предвижда от пет до седем години. Член 319в предвижда до седем години, когато деянието по чл. 319б е по отношение на данни, „които се дават по силата на закон, по електронен път или на електронен или друг носител“. Член 319г наказва въвеждането на компютърен вирус или на друга програма, предназначена за нарушаване на дейността на система или за узнаване, заличаване, изменение или копиране на данни без разрешение (до седем години). Член 319д наказва създаването и разпространението на програми, пароли и кодове за достъп с цел престъпление по чл. 171, ал. 3 или чл. 319а до 319г (до шест години). Член 319е се отнася до нарушение на чл. 6, ал. 2, т. 5 от Закона за електронния документ и електронните удостоверителни услуги [17].

SQL инжекция, използвана за достъп до данни без право, може да осъществи състава на чл. 319а, ал. 1. Когато чрез нея данни се копират, изменят или изтриват, възможна квалификация е чл. 319б, ал. 1, а при цел имотна облага и ал. 3. И двата члена се прилагат в немаловажни случаи. Това е авторов анализ на текста на закона, а не извод от съдебна практика.

1.3.4. Администраторът на данни: GDPR

Българският текст на Регламент (ЕС) 2016/679 (*GDPR*) не съдържа думата „инжекция“. Член 32, параграф 1 задължава администратора и обработващия да прилагат

подходящи технически и организационни мерки за сигурност, съобразени с риска и с „достиженията на техническия прогрес“, а буква г) включва процес на редовно изпитване и оценка на ефективността им. Член 33 изисква уведомяване на надзорния орган без ненужно забавяне и, когато е осъществимо, не по-късно от 72 часа след узнаването, освен ако няма вероятност нарушението да породи риск за правата и свободите на лицата. Според член 83, параграф 4, буква а) нарушенията на задълженията на администратора и обработващия съгласно членове 8, 11, 25 до 39, 42 и 43, към които спадат членове 32 и 33, подлежат на административно наказание „глоба“ или „имуществена санкция“ в размер до 10 000 000 евро или, в случай на предприятие, до 2 % от общия му годишен световен оборот за предходната финансова година, която от двете суми е по-висока [18]. Връзката с инжекциите е тълкувателна: когато за клас уязвимости съществува документирана и широко известна защита, нейната липса трудно може да се обоснове с достиженията на техническия прогрес. Така разсъждава *ICO* по случая *TalkTalk* по действалото тогава право [14]. Член 33 показва и друго: правото предвижда уведомяване *след* нарушението.

1.3.5. NIS 2 и Законът за киберсигурност

Директива (ЕС) 2022/2555 (*NIS 2*) задължава съществените и важните субекти да приемат подходящи и пропорционални мерки за управление на рисковете за мрежовите и информационните системи (член 21, параграф 1). Сред минималните мерки е „сигурност при придобиването на мрежови и информационни системи, разработване и поддръжка“, включително, в английския текст, „vulnerability handling and disclosure“ (параграф 2, буква д) [19]. Директивата се прилага към субекти от приложения I и II, които са най-малко средни предприятия, а в определени случаи независимо от размера (член 2) [19]. По данни на правна кантора и на проследяващ регистър в България тя е въведена чрез изменение на Закона за киберсигурност, прието на 5 февруари 2026 година, обнародвано на 13 февруари 2026 година и в сила от 17 февруари 2026 година [20, 21]. Заглавната бележка на пречистения текст на закона потвърждава само изменение, обнародвано в „Държавен вестник“, бр. 17 от 13-о число на 2026 година [22]. Съдържанието на закона по членове не е проверявано от първичен източник и не се цитира.

1.4. От наказание към превенция

Разгледаните норми действат след събитието: наказателният закон санкционира нападателя, регламентът изисква уведомяване след нарушението, а глобата на *ICO* е последваща. Нито една не намалява вероятността следващото приложение да съдържа същата слабост.

На 25 март 2024 година *CISA* и *FBI* публикуват съвместно предупреждение за проектиране с вградена сигурност (*Secure by Design Alert*) „Eliminating SQL Injection Vulnerabilities in Software“. То определя SQL инжекциите като устойчив клас дефекти, отбелязва, че индустрията знае как да ги елиминира от десетилетия (като пример: *MySQL* въвежда подготвените изрази през 2004 година), и призовава ръководителите на производителите да прегледат кода си и да изключат целия клас дефекти от настоящите и бъдещите продукти [3]. Основната препоръка е параметризирани заявки с подготвени изрази, които отделят SQL кода от данните на потребителя. Почистването на входа се оценява като крехко, трудно за налагане в голям мащаб и често заобиколимо. Препоръчва се параметризираните заявки да са стандартна практика, наложена в средата за разработка, например чрез библиотеки, които правят сигурния път пътя по подразбиране. Документът се позовава и на принципите за проектиране с вградена сигурност: поемане на отговорност за резултатите за клиентите и изкореняване на цели класове уязвимости, вместо отстраняване случай по случай [3].

Този аргумент е политически, но има техническа последица. Ако сигурният път трябва да бъде единственият, въпросът е какво може една библиотека да направи невъзможно. Предупреждението говори за SQL и за стойности на параметри. То не разглежда имената на таблици и колони, структурата на заявката и хранилищата, които не са релационни. Именно тук се намира изследователският въпрос на настоящата работа. Следващата глава разглежда какво вече е предложено като защита и докъде достига то.

II. ПРЕГЛЕД НА СЪЩЕСТВУВАЩИТЕ РЕШЕНИЯ

Предходната глава показа, че инжекциите са добре познат клас уязвимости, чрез който се извършват престъпления и който води до санкции за администраторите, и че политиката се измества от наказване към изключване на класа още при проектирането. Настоящата глава разглежда какви средства за това вече съществуват и къде всяко от тях спира. Първо се разглеждат защитите по слоеве, после академичната работа върху SQL и инжекциите извън SQL. Следва прегледът на библиотеките за типово безопасно конструиране на заявки. Главата завършва със сравнителна таблица и с формулировка на пропуска, който следващите глави адресират.

Прегледът се основава на документацията на проектите и на оригиналните публикации. Когато в прочетените страници даден факт не е посочен, в таблиците е записано „н.д.“ (не е посочено). Авторът не е стартирал разглежданите библиотеки, затова твърденията за тях са твърдения на документацията, а не измерване.

2.1. Защити по слоеве

Основният практически източник е наръчникът на *OWASP* за предотвратяване на SQL инжекции [2]. Той поставя на първо място подготвените изрази с параметризирани заявки, следвани от съхранените процедури и проверката по списък на допустимите стойности. Екранирането е определено като „STRONGLY DISCOURAGED“, а към защитите се добавят най-малките привилегии. За всяка защита по-долу се разглеждат две неща: какво гарантира и как се проваля.

Екраниране е преобразуване на специалните знаци във входа така, че интерпретаторът да ги третира като данни. Наръчникът настоява ръчното екраниране да не замества свързването на параметри, защото правилата зависят от диалекта и от конфигурацията на базата [2]. Гаранцията е локална: важи за конкретния диалект и място, където екранирането е извършено, и се губи на всяко пропуснато място. Екранирането остава практика там, където параметри не могат да се ползват: ръководството на *Neo4j* препоръчва за идентификаторите в обратни апострофи тези знаци да се удвояват [23].

При **параметризираните заявки** текстът и стойностите се предават поотделно: заявката съдържа маркери, а драйверът свързва стойностите към тях. Стойността не може да промени структурата, защото тя вече е разчетена. Това е защитата, препоръчана от

OWASP, *CISA* и *FBI* [2, 3]. Същият принцип стои зад *Neo4j*, където планът на заявката е компилиран преди подставянето на стойностите [23]. В *Cassandra* заявка с поне един маркер трябва да бъде подготвена; документацията на *CQL* описва това като оптимизация и не го свързва със сигурността [24]. Границата е посочена в наръчника на *OWASP*: маркерите заместват стойности, а не имена на таблици и колони и не ключови думи [2]. Документацията на *EF Core* казва същото [25]. Защитата не се прилага автоматично: ако заявката е сглобена с конкатенация, параметри няма. Тя не отговаря и на въпроса дали потребителят има право да види даден запис.

Съхранените процедури пренасят заявките в базата. Наръчникът предупреждава, че свързването на аргументите защитава извикването, но не и динамичния SQL, който процедурата сглобява вътре в себе си [2]. Халфонд, Виегас и Орсо описват същото като разпространено заблуждение [26]. Гаранцията е условна: зависи от начина, по който е написана процедурата.

Проверката по **списък на допустимите стойности** (*allow-list*) се препоръчва за структурни избори, които не могат да се параметризират. Пример е посоката на сортиране: входът се съпоставя с литералите `ASC` или `DESC`, притежавани от приложението. Проверката допълва свързването на параметри, но не го замества [2]. Гаранцията е силна само когато множеството може да се изброи и не помага при свободен текст.

Най-малките привилегии ограничават какво може да направи вече променена заявка. *OWASP* предупреждава приложението да не получава права на администратор на базата [2], а *Neo4j* препоръчва контрол на достъпа по роли [23]. Мярката не предотвратява инжекцията, а намалява последиците ѝ.

Защитната стена за уеб приложения (*web application firewall*, *WAF*) проверява HTTP трафика пред приложението. *ModSecurity* е отворен механизъм за такива стени за *Apache*, *IIS* и *Nginx* [27], а *OWASP Core Rule Set* (*CRS*) е наборът правила за него. Всяко съвпаднало правило добавя точки към резултата на заявката, която се отхвърля при праг, по подразбиране 5 [28]. По-високите от четирите нива на параноя откриват повече атаки, но дават повече фалшиви тревоги, които налагат изключения на правила и могат да доведат до изключване на *CRS* [29]. Стената вижда HTTP заявката, а не заявката към базата, която приложението по-късно ще сглоби. Гаранцията ѝ е вероятностна и не променя причината за уязвимостта.

Защитите се разделят на две групи. Екранирането, параметрите и списъкът на

допустимите стойности действат върху самата заявка и дават силна гаранция, но само за онази част, за която са приложими, и само когато програмистът ги приложи. Привилегиите и защитната стена действат извън заявката и ограничават вредата или вероятността. Общото между всички е, че се прилагат от човек на всяко място поотделно.

2.2. Академична работа по откриване и предотвратяване

Су и Васерман определят инжекцията формално: за да успее атака, входът, който попада в заявката, трябва да променя нейната предвидена синтактична структура. Според тях това е първото формално определение на атаките чрез инжектиране на команди в уеб приложенията. Заедно с него авторите дават алгоритъм за предотвратяването им, основан на контекстно-свободни граматики. Реализацията им *SQLCHECK*, оценена върху реални приложения със системно събрани реални атаки, по думите им не е дала нито фалшиво положителни, нито фалшиво отрицателни резултати [4]. Определението се използва и в настоящата работа (глава III).

Халфонд, Виегас и Орсо правят обзор на известните видове SQL инжекции (сред тях тавтологии, обединени и натрупани заявки, атаки чрез съхранени процедури, алтернативни кодировки) и оценяват съществуващите техники спрямо тях. Основното им наблюдение е, че много предложени решения адресират само подмножество от проблема [26]. Техниката *AMNESIA* на Халфонд и Орсо строи чрез статичен анализ модел на законните заявки на приложението и проверява всяка генерирана заявка спрямо него по време на изпълнение. Върху седем приложения тя е спряла всички опитани атаки без фалшиво положителни резултати [30]. *CANDID* на Бандхакави и съавтори е, според резюмето, схема за автоматично преобразуване на уеб приложения, така че по твърдение на авторите те да са безопасни срещу SQL инжекции. Методът е посочен в заглавието като динамична оценка на кандидати [31]. *SQLrand* на Бойд и Керомитис рандомизира набора от инструкции на SQL, така че инжектираният текст да е невалиден за парсера, а междинен прокси превежда рандомизирания език към стандартния за *MySQL* [32].

Тези работи имат общи черти. Всички са насочени към SQL. Действат върху вече построена заявка: сравняват я с модел, с кандидат или с рандомизиран речник, а не променят начина, по който се построява. Изискват допълнителен инструмент: анализ, модифицирана среда или прокси. Гаранцията се налага по време на изпълнение върху вече построената заявка, а не при построяването ѝ. Някои от работите дават и формално

обосновани алгоритми (така Су и Васерман претендират за алгоритъм, който е коректен и пълен), но оценката им е върху подбрани приложения. Определението на Су и Васерман остава общата основа, спрямо която се измерва успехът на всички тях.

2.3. Инжекции извън SQL

Литературата опровергава мнението, че хранилищата без SQL са имунизирани. Рон, Шулман-Пелег и Бронщайн представят техники за инжектиране в NoSQL хранилища, сред които инжектиране чрез масиви в *PHP*, *OR* инжектиране в *MongoDB* и инжектиране на произволен *JavaScript*. В един пример с оператора `$ne` условията за потребителско име и парола се изпълняват за всяка стойност, различна от посочената [33]. Част от същите автори публикуват статия в *IEEE Security & Privacy*, според резюмето на която само анализът на кода е недостатъчен за предотвратяване на такива атаки и мерките трябва да обхващат целия жизнен цикъл на софтуера [34]. Ръководството за тестване на уеб сигурността на *OWASP* (версия 4.2) обяснява, че атаките срещу NoSQL могат да се изпълняват в процедурен език, затова въздействието им може да е по-голямо, и че филтрирането на знаци, специални за HTML, не защитава JSON интерфейс. В *MongoDB* интерфейсът очаква BSON и предоставя безопасно сглобяване на заявки, но несериализираният JSON и изразите на *JavaScript* се допускат в алтернативни параметри, като най-често използваният е операторът `$where` [35].

Документацията на *Neo4j* назовава Cypher инжекцията „братовчед“ на SQL инжекцията и посочва като основна защита подаването на входа като параметри. Конкатенирането на вход в низ, подаван на `apoc.cypher.doIt`, остава уязвимо, а етикетите, типовете на връзките и имената на свойства според статията не могат да се подават като параметри и затова се препоръчва да се избягват или санитизират [23]. Текущото ръководство по Cypher уточнява, че параметри не могат да заместват директно ключове на свойства, типове връзки и етикети (например `n:$param` е невалидно), но етикетите и типовете могат да се задават динамично чрез израз [36]. Следователно в Cypher имената остават избираеми по време на изпълнение по документиран път.

Документацията на *Redis* приема, че сървърът е за доверени клиенти в доверена среда и че недоверен достъп трябва да се опосредства от слой, проверяващ входа. За инжекциите тя твърди, че протоколът няма понятие за екраниране на низове, че „injection is impossible under normal circumstances using a normal client library“ и че протоколът

използва низове с префикс за дължина, безопасни за двоични данни. Същото важи за скриптовете на *Lua*, но тялото на скрипт не бива да се сглобява от недоверени низове [37]. Условието „нормална клиентска библиотека“ означава, че гаранцията на протокола не е гаранция на приложението. За *Cassandra* документацията на CQL описва маркерите и подготвените изрази, но в прочетените страници не е открито самостоятелно ръководство за инжекции в CQL [24].

Всяко хранилище решава проблема със свои средства: параметри, подготвени изрази, протокол с префикс за дължина. Стойностите са защитени по замисъл на протокола или интерфейса, но точно там, където приложението избира структура или име, документацията предупреждава за граница. Гаранциите са формулирани поотделно от производителя на всяко хранилище. В прегледаните източници няма общо определение, което да обхваща всички хранилища, а академичните работи от предходния раздел не покриват нито едно от тях.

2.4. Типово безопасно конструиране на заявки

Друг подход е заявките да не се пишат като низове, а да се изграждат от типове и функции на езика. Така компилаторът проверява имената и типовете. Разглеждат се *LINQ* с *EF Core* (C#), *jOOQ* (Java), *Diesel* и *sqlx* (Rust), *esqueleto* и *opaleye* (Haskell) и *sqlpp11* (C++). За всяка се търси какво прави безопасно и дали остава път към суров текст или към идентификатори, избрани по време на изпълнение.

Според документацията на *EF Core* методите `FromSql` и `FromSqlInterpolated` са безопасни, защото предават данните като параметър, а `FromSqlRaw` може да е уязвим. Когато името на колона трябва да се вмъкне динамично, се ползва `FromSqlRaw`, а санитизирането е отговорност на програмиста [25]. В *jOOQ* SQL обикновено се строи като типизирано дърво със свързани стойности и според документацията по този път не е възможно да се въведат инжекции. Изключение са методите за суров SQL, отбелязани с `org.jooq.PlainSQL` от версия 3.6 [38], чието използване документацията свързва с проверка чрез *Checker Framework* или *Error Prone* [39]. *Diesel* заявява стремеж да елиминира неправилните взаимодействия с базата на етапа на компилацията [40] и според документацията изисква схемата да е описана в кода [41]. Суров SQL се пише чрез `sql_query`, а малки фрагменти чрез `sql`; страницата на `sql_query` не споменава инжекциите и предупреждава само за типовете [42]. Макросът `query!` на

sqlx проверява SQL низа при компилация срещу жива база или запазен кеш [43]. Във версия 0.9.0 функцията `sqlx::query` приема `impl SqlSafeStr`, динамично построен низ трябва изрично да се обвие в `AssertSqlSafe`, а документацията предупреждава срещу `format!` [44].

В *esqueleto* има `rawSql` с текст и списък от стойности, а SQL функция може да се въведе чрез `unsafeSqlFunction` или `unsafeSqlBinOp` [45]. В *opaleye* функциите за таблица и колона приемат обикновен низ: `table :: String -> ...` и `tableField :: String -> ...` [46], а модулът съдържа и `unsafeCast :: String -> ...` [47]. Това не доказва, че недоверени имена могат да бъдат подадени, но показва, че типовата система не го забранява. *sqlpp11* е типово безопасен вграден език за SQL в C++, чиито таблици се пораждат от DDL и чиито заявки се проверяват при компилация за грешки в синтаксиса, типовете и имената [48]. За неподдържани изрази документацията описва `sqlpp::verbatim` [49]. Същият автор поддържа и *sqlpp23*, чиято документация описва `verbatim_table`, който добавя таблици, дори когато те не са известни при компилация, както и `verbatim` и `verbatim_clause` [50, 51]. Това е най-близката прегледана библиотека на C++ до подхода на настоящата работа и затова е важно, че и в нея път към суров текст е предвиден по дизайн. Преглед, изчерпателен за всички библиотеки на C++, не е правен.

Таблица 1 показва повтарящ се модел. Защитата на стойностите е нормален случай там, където документацията я посочва. Идентификаторите и структурата са защитени дотолкова, доколкото програмистът остава в типизирания интерфейс. Във всяка прегледана библиотека обаче е документиран път към суров текст. Обозначен е различно: наставката `Raw` в *EF Core*, анотацията `@PlainSQL` с възможност за проверка в *jOOQ*, явното `AssertSqlSafe` в *sqlx*. Тези обозначения помагат при преглед, но налагането им е конвенция или външен инструмент, а не свойство на типовата система. В *opaleye* именуването чрез низове е част от обикновения интерфейс. Таблицата не казва, че библиотеките са небезопасни, а че гаранцията им зависи от това, че изходът няма да бъде използван.

2.5. Сравнение на подходите

Таблица 2 съпоставя разгледаните подходи по критерии, които следват от определението на Су и Васерман [4]: кои части на заявката се защитават, покриват ли

Таблица 1. Типово безопасни библиотеки: какво защитават и къде остава изход

Библиотека	Стойности	Идентификатори и структура	Път към суров текст	Проверка
EF Core (LINQ)	Параметри при `FromSql` [25]	Имена на колони не се параметризират, нужен е `FromSqlRaw` [25]	`FromSqlRaw`, `SqlQueryRaw`, `ExecuteSqlRaw`	Типове при компилация, превод в SQL при изпълнение
jOOQ	Свързани стойности в дървото [38]	Типизирано дърво [38]	Методи с `@PlainSQL` [38, 39]	Компилация (Java), по желание допълнителна проверка
Diesel	Свързани стойности [52]	Схемата е описана в кода [41]	`sql_query`, `sql`	Компилация
sqlx	Свързани параметри [44]	`query!` проверява литерален низ срещу база [43]	`AssertSqlSafe` върху динамичен низ [44]	Компилация срещу жива база или кеш
esqueleto	`rawSql` приема стойности, за DSL н.д. [45]	н.д.	`rawSql`, `unsafeSqlFunction`, `unsafeSqlBinOp` [45]	Компилация (Haskell)
opaleye	н.д.	Имена като `String` [46]	`unsafeCast` с низ [47]	Компилация (Haskell)
sqlpp11 / sqlpp23	В sqlpp11 подготвени изрази с `parameter()` [53]	В sqlpp11 таблици от DDL [48], в sqlpp23 е възможен `verbatim_table`	`verbatim`, `verbatim_table`, `verbatim_clause` [49, 50]	Компилация (C++)

се хранилища извън SQL, кога се налага гаранцията и има ли изход. Оценките са на автора, въз основа на цитираните по-горе източници. „Частично“ означава, че гаранцията важи при определени условия, посочени в съответния раздел.

Таблица 2. Сравнение на подходите по критерии

Подход	Стойности	Идентификатори	Структура	Не-SQL хранилища	Кога се налага	Изход
Екраниране	частично	частично	не	зависи от диалекта	изпълнение	всяко пропуснато място
Подготвени изрази	да	не	не	да, където има маркери	изпълнение	сглобяване на низ
Съхранени процедури	при параметри вътре	не	не	н.д.	изпълнение	динамичен SQL вътре
Списък на допустимите	да, за изброими	да, за изброими	да, за изброими	да	изпълнение	свободен текст
Най-малки привилегии	ограничава вредата	ограничава вредата	ограничава вредата	да, като принцип	внедряване	не е приложимо
Защитна стена	по вероятност	по вероятност	по вероятност	според правилата	внедряване	пропуснати атаки
Откриване по структура (SQLCHECK, AMNESIA, CANDID, SQLrand)	да	да	да	не, само SQL	изпълнение, с инструмент	само SQL в оценките
Типови езици за заявки	да	зависи от библиотеката	да, без суров текст	основно SQL	компилация	да, във всички прегледани

Таблица 2 позволява три наблюдения. Първо, никой подход не съчетава защита на стойностите, идентификаторите и структурата, покритие на хранилища извън SQL и липса на изход. Параметрите покриват стойностите, но по природа не покриват имената. Откриването по структура покрива всички части на SQL заявката, но е насочено само към SQL и действа върху вече построена заявка. Типовите езици за заявки са единствените, които налагат гаранцията при компилация, но във всички прегледани случаи оставят път към суров текст. Второ, подходите, които действат при изпълнение или внедряване, откриват или блокират проблема, след като кодът вече съществува, а подходите при компилация могат да предотвратят самото изграждане на небезопасната заявка, но гаранцията им е толкова силна, колкото е затворен изходът. Трето, ако изходът е част от гаранцията, нито един прегледан подход не дава гаранция за целия клас, а само за частта,

която съответства на неговия механизъм. Тази разлика между защита на отделните случаи и изключване на класа е същността на аргумента на *CISA* и *FBI* (раздел 1.4).

2.6. Пропускът

В рамките на прегледаната литература и документация могат да се направят три наблюдения. Първо, работата по откриване и предотвратяване на инжекции се концентрира върху SQL [4, 26, 30, 31, 32]. Работата върху NoSQL е по-малко и описва техники и мерки за смекчаване [33, 34], а документацията за Neo4j, Redis и Cassandra е формулирана отделно за всяко хранилище. Второ, типовите езици за заявки защитават стойностите, но в прегледаните библиотеки запазват начин да се подаде идентификатор или текст на заявката по време на изпълнение. Той е обозначен по различен начин, но е налице. Трето, *SQLCHECK* и *AMNESIA* са измерени върху реални приложения, но само за SQL [4, 30]. В прегледаните източници не е открита оценка, която прилага гаранция, наложена при компилация, към няколко разнородни хранилища и я сравнява с измерена уязвима базова реализация.

Пропускът е следователно в съчетанието на три условия, а не във всяко от тях поотделно: гаранция, наложена при компилация, обхващаща няколко разнородни хранилища и оценена спрямо измерена уязвима базова реализация. Формулировката е ограничена до прегледаната литература и не е твърдение за абсолютна новост. Следващите глави адресират този пропуск: глава III въвежда понятията, а останалите разглеждат подхода, неговата реализация и оценката му.

III. ТЕОРЕТИЧНА ОСНОВА

Предходната глава позиционира работата спрямо съществуващите защити. Настоящата глава въвежда само понятията, необходими за разбирането на собствения принос: формално определение на инжекцията, трите принципно различни начина за нейното предотвратяване, разликите между езиците за заявки на изследваните хранилища и ролята на времето на компилация като граница на доверие. Последната точка е мостът между определението и реализацията, разгледана в глава V.

3.1. Инжекцията като промяна на синтактичната структура

Интуитивното описание на инжекцията („въведеният текст се изпълнява като код“) е недостатъчно точно, за да служи за критерий при оценка. В настоящата работа се използва определението на Су и Васерман [4]. Заявката се разглежда като изречение от формален език и съответно като синтактично дърво. Според Су и Васерман входът, попаднал в заявката, трябва да заема цял възел на дървото от допустим вид, като допустимите видове се задават от политика. Тук е избрана най-строгата политика: всеки фрагмент от заявката, произлизащ от недоверен вход, трябва да попада изцяло в едно листо на дървото, тоест да бъде стойност. **Инжекция** е налице тогава, когато недоверен вход променя синтактичната структура на заявката: когато той обхваща повече от едно листо или въвежда нови вътрешни възли.

Определението има две предимства. Първо, то е независимо от конкретния полезен товар (*payload*): то не изисква списък от опасни символи, а критерий за структурна цялост. Второ, то е независимо от езика. Едно и също понятие обхваща SQL, документните заявки на MongoDB, Cypher, CQL и командния протокол на Redis, стига за всеки от тях да е известна граматиката, по която хранилището разбира заявката. Именно тази независимост позволява изследователският въпрос да бъде поставен за цяло семейство от класове инжекции, а не за всеки поотделно.

Класификацията на Халфонд, Виегас и Орсо [26] допълва определението от страна на практиката. Тя описва видовете атаки (тавтологии, нелогични или некоректни заявки, обединения чрез `UNION`, натрупани заявки, съхранени процедури, извеждане на информация чрез слепи атаки по истинност и по време, алтернативни кодировки), начините на въвеждане (потребителски вход, бисквитки, сървърни променливи, инжекция

от втори ред) и намеренията на нападателя (например извличане на данни, заобикаляне на удостоверяване, промяна на данни, отказ от услуга). Корпусът от полезни товари, използван в глава VII, е подреден по видове атаки.

3.2. Три начина за отделяне на данните от кода

Общата причина за всички инжекции е една: данни и код се пренасят по един и същ канал, най-често като един низ, и хранилището ги разделя едва при синтактичния анализ. Всяка защита следователно отговаря на въпроса как недоверената стойност да остане листо в дървото. Принципно се различават три подхода.

Екраниране (*escaping*). Стойността остава в същия канал, но се преобразува така, че синтактичният анализатор да не я разпознае като граница на листото. Сигурността зависи от пълното и правилно преобразуване за конкретния диалект и кодиране на символите. Пропуск на един специален символ или разлика между представата на приложението и на хранилището за кодирането е достатъчна, за да се загуби гаранцията.

Свързване на параметри (*parameter binding*). Заявката се изпраща с маркери на местата на стойностите, а самите стойности пътуват отделно, по протокол, който хранилището никога не подлага на синтактичен анализ като заявка. Стойността не може да стане вътрешен възел, защото не участва в анализа изобщо. Това е механизмът на подготвените заявки [2].

Конструиране (*construction*). Структурата на заявката не се сглобява от низове, а се изразява в средство, в което недоверените данни нямат път към структурата. Свързването защитава стойностите, но оставя отворен въпроса кой определя структурата. Конструирането отговаря на този въпрос.

Разграничението е съществено, защото свързването не покрива всички части на заявката. Нито едно от изследваните хранилища не позволява параметър на мястото на име на таблица или колона. Затова приложения, които използват подготвени заявки за стойностите, нерядко сглобяват идентификаторите като низове, и именно там остава уязвимостта. Настоящата работа изследва конструирането като допълнение към свързването, което затваря тази празнина.

3.3. Езиците на изследваните хранилища

Определението от точка 3.1 предполага граматика. Петте класа инжекции в обхвата на работата се различават именно по това каква граматика анализира хранилището и дали изобщо анализира текст, получен от приложението.

Таблица 3. Граматика и клас инжекция при изследваните хранилища

Хранилище	**Какво се анализира**	**Клас инжекция**
PostgreSQL, MySQL, SQLite	текст на SQL	SQL инжекция: тавтологии, `UNION`, натрупани заявки, слепи атаки
Cassandra	текст на CQL	CQL инжекция, сходна със SQL, но с по-беден език
Neo4j	текст на Cypher	Cypher инжекция, включително извикване на процедури
MongoDB	документ BSON, не текст	инжекция на оператори: стойност, която се интерпретира като `$ne` или `$where`
Redis	кадри RESP с префикс за дължина	инжекция на протоколно ниво чрез `CR LF`, ако командата се сглобява като текст

Таблица 3 показва, че хранилищата не са еднородни по отношение на заплахата. При SQL, CQL и Cypher хранилището анализира текст и опасността е класическата. При MongoDB заявката е структуриран документ и инжекцията се изразява по друг начин: стойност, която приложението очаква да е низ, пристига като обект с ключ, започващ с `$`, и се интерпретира като оператор. При Redis протоколът RESP предава всеки аргумент с изрично указана дължина. Ако клиентът кодира аргументите правилно, съдържанието на аргумента не може да бъде прочетено като граница на команда, независимо от символите в него. Следователно за две от седемте хранилища правилното кодиране премахва цялата граматика, от която атакуващият би могъл да излезе, а не само я екранира.

3.4. Времето на компилация като граница на доверие

Конструирането от точка 3.2 изисква средство, в което недоверените данни нямат път към структурата. В езика C++ такова средство е разграничението между стойностите, известни по време на компилация, и тези, известни едва по време на изпълнение.

Аргументите на шаблон, които не са типове (*non-type template parameters*), трябва да бъдат **константни изрази**. Стойност, която зависи от входа на програмата, по дефиниция не е константен израз и не може да бъде такъв аргумент. Ако следователно всеки елемент на синтактичната структура на заявката (име на таблица, име на колона, вид на операцията, оператор на сравнение, наличие на клауза) е закодиран като аргумент на шаблон или като тип, то никоя стойност, получена по време на изпълнение, не може да го промени. Входът може да достигне заявката само там, където типовата система изрично го допуска.

Аргументът може да се формулира като свързване на определението с типовата система:

1. Нека всички вътрешни възли на синтактичното дърво на заявката са определени от аргументи на шаблони и типове. Тогава те са фиксирани в момента на компилация.

2. Нека единственият път, по който стойност по време на изпълнение достига заявката, е типизиран заместител, който се свързва като параметър съгласно точка 3.2.

3. Тогава недоверен вход може да заеме само листо и не може да промени структурата. Съгласно определението на Су и Васерман [4], приложено с избраната политика, инжекция не е възможна.

Двете предпоставки не се приемат на доверие. Първата се проверява в глава VII чрез програми, които се опитват да подадат стойност по време на изпълнение на мястото на идентификатор или структура и трябва да бъдат отхвърлени от компилатора. Втората се проверява чрез диференциален тест и чрез анализа на всеки конектор в глава V. Именно проверката на втората предпоставка разкри случай, в който тя не е била изпълнена: казусът в глава VI.

Цената на подхода е симетрична на гаранцията. Ако идентификаторите и структурата не могат да зависят от входа, то и програмистът не може да ги направи зависими. Схеми, които се определят динамично, не се поддържат. Това не е пропуск, а същността на гаранцията, и е обсъдено като ограничение в глави IV и в заключението.

IV. МОДЕЛ НА ЗАПЛАХИТЕ И МЕТОД

Теоретичната основа от глава III показа при какви предпоставки конструирането по време на компилация изключва инжекцията. За да бъде това твърдение проверимо, е необходимо да се определи срещу какъв нападател се твърди гаранцията, какво точно се защитава и по какви критерии ще се прецени успехът. Настоящата глава фиксира тези три неща преди представянето на каквито и да било резултати и описва метода, по който те се проверяват.

4.1. Защитаван актив

Защитаваният актив е **структурата на командата**, която приложението изпраща към хранилището за данни. Стойностите в една заявка са данни, а структурата определя какво хранилището е упълномощено да върне или промени. Нападател, който превръща `WHERE id = $1` в `WHERE id = $1 OR 1=1`, не е променил данни. Той е променил обхвата на правомощията на заявката. Всеки клас инжекция от таблица 3 е разновидност на този ход.

4.2. Модел на нападателя

Нападателят контролира вход, който приложението използва при изграждането на заявка. Той не контролира изходния код на приложението, компилатора и драйвера на хранилището. Разграничават се три нива по нарастваща сила. Разграничението е съществено, защото обичайното обсъждане на инжекциите ги слива.

Ниво 1: нападателят контролира стойности. Стойност от формуляр, заглавна част на заявка или съобщение достига заявката. Това е обичайният случай и единственият, който свързването на параметри покрива.

Ниво 2: нападателят контролира идентификатори. Нападателят избира име на таблица, на колона или ключ за подреждане. Тук свързването на параметри не помага, защото нито едно от изследваните хранилища не позволява параметър на мястото на идентификатор.

Ниво 3: нападателят контролира структурата. Нападателят определя формата на заявката, например кои условия участват в клаузата `WHERE`, чрез построител на филтри, който сглобява предикати според заявката.

Гаранцията на изследваната библиотека се твърди и за трите нива. На ниво 1 тя разчита на свързване, а на нива 2 и 3 на това, че съответната възможност изобщо не е изразима по време на изпълнение (точка 3.4).

4.3. Извън обхвата

Следните заплахи съзнателно са извън твърдяната гаранция и се обсъждат като ограничения, а не като пропуски:

- **инжекция от втори ред**: стойност, прочетена от хранилището и използвана от приложението като идентификатор другаде, е извън видимостта на библиотеката;
- **логически грешки в приложението**: липсваща проверка на правомощия или прекалено широк филтър не са инжекция;
- **операции по схемата** (*DDL*) и миграции, които изискват отделен анализ;
- **уязвимости в драйверите** (`libpq`, `hiredis`, `bsoncxx` и останалите), които се приемат за коректни;
- **отказ от услуга** чрез скъпи, но легитимни заявки, който е въпрос на капацитет, а не на инжекция.

4.4. Изследователски въпрос и хипотеза

Работата отговаря на въпроса:

> *Може ли една абстракция, конструираща заявките по време на компилация, да елиминира цяло семейство от класове инжекции при хетерогенни хранилища за данни, и къде точно спира тази гаранция?*

Хипотезата е, че при изпълнени предпоставки от точка 3.4 никой полезен товар, подаден през документирания интерфейс, няма наблюдаем ефект върху хранилището, докато същият полезен товар, подаден в наивно сглобена заявка, има. Втората част на въпроса, къде спира гаранцията, е равностойна на първата. Работа, която отчита само успехи, не позволява да се прецени валидността на твърденията си.

4.5. Критерии за успех

Критериите са определени преди измерванията и не са променяни след тях.

Таблица 4. Критерии за успех

№	Критерий
К1	Никой полезен товар от корпуса, подаден през документирания интерфейс, не предизвиква наблюдаем ефект: върнати редове, които не би трябвало да съществуват, промяна в данните или схемата, или принудително изпълнение.
К2	Всеки опит идентификатор или структура на заявката да зависи от стойност по време на изпълнение се отхвърля от компилатора, и то по причината, която се проверява, а не по случайна друга.
К3	Наивната заявка, сглобена от същите полезни товари, е компрометирана поне веднъж при същата постановка. Без това изпълнението на К1 не доказва нищо.
К4	Границите на гаранцията са назовани и, където е възможно, демонстрирани.

Критерий К3 заслужава коментар. Сравнение, при което нито защитеният, нито незащитеният път са засегнати, не различава защитата от неефективен корпус. Наивната заявка е съзнателно уязвима и служи за база, спрямо която резултатът на библиотеката придобива смисъл. Ако тя не бъде компрометирана, тестът се отчита като неуспешен.

4.6. Метод на оценката

Методът съчетава три независими източника на доказателства, всеки от които покрива различна предпоставка.

Отрицателни компилационни тестове (за К2). Всеки тест е програма, която се опитва да направи идентификатор или структура зависими от стойност по време на изпълнение и трябва да бъде отхвърлена от компилатора. Такъв тест сам по себе си е слаб: той би преминал и при грешка, несвързана с проверяваното свойство, например липсващ заглавен файл. Затова се прилагат две мерки. Първо, всяко отхвърляне трябва да съответства на образец за очакваната диагностика. Второ, наборът съдържа **контролни случаи**, които използват същата постановка и трябва да се компилират. Ако постановката

е повредена, контролните случаи също се провалят.

Диференциално тестване (за К1 и К3). Всеки приложим полезен товар се подава по два пътя, които задават един и същ въпрос към една и съща база данни: през наивно сглобена заявка и през библиотеката. За всяко изпълнение се използва нова база данни. Резултатът се определя от **оракул**, който наблюдава само ефекти: върнати редове, промяна на състоянието и принудително прекратяване на изпълнението.

Анализ на изходния код (за предпоставка 2 от точка 3.4 и за К4). За всеки от седемте конектора се установява как точно стойността достига драйвера. Номерата на редовете, посочени в изложението, се отнасят към версия `a17f4e7` на хранилището с изходния код.

4.7. Какво методът не доказва

Методът има граници, които се посочват предварително, за да не се надценяват резултатите.

- Оракулът вижда само ефекти. Сляпа атака, която променя логиката на заявката, но връща нула редове, не се отчита като компрометиране. Следователно броят на компрометиранията на наивния път е **долна граница**. Това изместване работи срещу твърдението на работата, а не в негова полза.
- Диференциалният тест използва една форма на заявка за хранилище. Друга форма би могла да позволи други атаки срещу наивния път.
- Корпусът е краен. Отсъствие на успешна атака срещу библиотеката не е доказателство за отсъствие на всякаква атака. Доказателството за това се търси в аргумента от точка 3.4 и в компилационните тестове, а не в корпуса.
- Компилационните тестове се изпълняват с GCC и Clang. Поведението на други компилатори не е проверено.

V. РЕАЛИЗАЦИЯ НА ГАРАНЦИЯТА ПРИ СЕДЕМ ХРАНИЛИЩА

Моделът на заплахите от глава IV изисква гаранцията да се провери поотделно за всяко хранилище, защото предпоставка 2 от точка 3.4 се изпълнява от конкретния конектор, а не от общата абстракция. Настоящата глава описва накратко платформата, установява откъде идва всяка част на заявката и проследява за всеки от седемте конектора как стойността достига драйвера. Номерата на редовете се отнасят към версия `a17f4e7` на хранилището с изходния код.

5.1. Платформата

Изследваната библиотека е разработена в предходна дипломна работа за ОКС „Бакалавър“ [5], в която са описани обектният модел, междинното представяне на заявките, асинхронното изпълнение и конекторите. Настоящата работа не преповтаря тези резултати и не претендира за тях. Тя използва библиотеката като платформа и изследва въпрос, който предходната работа не поставя: доколко конструирането по време на компилация предотвратява инжекциите и къде спира тази защита. Приносите, формулирани в заключението, се отнасят само към това изследване и към промените в библиотеката, произтекли от него.

За изложението са необходими три понятия от платформата. **Свойство** (`orm::property<T, "name">`) описва колона: типът на стойността и името на колоната са аргументи на шаблона. **Междинното представяне** на заявката (*IR*) е стойност от тип като `select_query<...>`, чиито параметри на шаблона кодират избраните колони, условията и ограниченията. **Конекторът** превежда междинното представяне към езика на конкретно хранилище и предава стойностите на драйвера.

5.2. Откъде идва всяка част на заявката

Листинг 1 показва заявка, написана по документирания начин.

```
struct Item
{
    orm::property<int, "id">           id;    // името на колоната е аргумент на шаблон
    orm::property<std::string, "name"> name;
};

namespace orm
```

```
{
    template <>
    struct table_name_trait<Item>
    {
        static constexpr std::string_view value = "items";   // константен израз
    };
}

constexpr auto q = orm::select(orm::field<&Item::id>)
                       .where(orm::field<&Item::name> == orm::Placeholder<std::string>{});

auto rows = db.execute(q, input);   // стойността пътува отделно от заявката
```

Listing 1: Заявка, написана по документирания начин

В листинга трябва да се забележат три неща: името на колоната е част от типа на свойството, името на таблицата е константен израз, а стойността `input` не участва в изграждането на `q` и се подава едва при изпълнението. Таблица 5 обобщава произхода на всяка част.

Таблица 5. Произход на частите на заявката

Част на заявката	**Произход**	**Достъпна ли е за вход по време на изпълнение**
Имена на таблици и колони	`constexpr` метаданни на указател към член (`details/member_pointer.hpp:44`, `entity/property.hpp:19`)	Не
Форма на заявката (избор, условие, съединение, подреждане)	типове в междинното представяне	Не
Стойности	типизирани заместители `Placeholder<T>` и `ph<T, _N>`	Да, само като данни

Таблицата съдържа твърдение, което не следва автоматично от синтаксиса на листинга: че няма *друг* път, по който стойност да достигне заявката. Това твърдение

се оказа невярно за предходната версия на библиотеката и е обект на казуса в глава VI.

5.3. Механизми при отделните хранилища

Гаранцията е една и съща за всички хранилища, но механизмът, по който тя се постига, не е. Всеки конектор използва най-силното средство, което драйверът му предоставя. Таблица 6 показва резултата от анализа на изходния код.

Таблица 6. Как стойността достига драйвера при всеки конектор

Хранилище	Механизъм	Вид	Място в кода
PostgreSQL	`PQexecParams` с масиви от параметри	свързване в сървъра	`postgresql_live.hpp:552, 615`
MySQL	подготвени заявки с `MYSQL_BIND`	свързване в сървъра	`mysql_live.hpp:316`
SQLite	`sqlite3_prepare_v2` и `sqlite3_bind_*`	свързване	`sqlite_db.hpp:62, 67, 72`
Cassandra	`cass_statement_bind_*`	свързване в сървъра	`cassandra_live.hpp:200-221`
Neo4j	параметри на Bolt чрез `neo4j_map_entry`	типизирани параметри на драйвера	`neo4j_live.hpp:174`
Redis	форми с форматиращ низ и `argv` на `hiredis`	кодиране на протоколно ниво	`redis_live.hpp:109, 180, 257`
MongoDB	построител на документи `bsoncxx`	структурно, без текстов език	`mongodb_live.hpp:214-238`

От таблицата се извеждат четири вида механизъм. Четири конектора използват свързване на параметри. Neo4j използва типизирани параметри на драйвера. Redis разчита на кодирането на протокола. MongoDB изобщо не изгражда текст. Изводът е, че една и

съща абстракция се защитава пред четири различни протокола и аргументът за сигурност трябва да е валиден поотделно за всеки от тях.

5.3.1. Хранилища без текстов език за заявки

При MongoDB и Redis стойността никога не става част от текст, който хранилището анализира синтактично. MongoDB получава документ BSON, в който стойността е поле със собствен тип. Redis получава кадри RESP, в които всеки аргумент е предшестван от дължината си. Липсва граматика, от която низ на нападателя би могъл да излезе. Тази позиция е по-силна от свързването на параметри, а не равностойна на него, тъй като не зависи от правилно разделяне на маркери и стойности. Разликата обикновено се губи, когато инжекциите в NoSQL се разглеждат като допълнение към SQL инжекциите.

Тази позиция е валидна само докато стойността не се интерпретира като оператор. При MongoDB конекторът добавя стойността като типизирано поле на документа (`mongodb_live.hpp:214-238`), а не разчита произволен документ от входа. Затова низ, който съдържа обект с оператора `$ne`, остава низ.

5.3.2. Neo4j: форматиране, което не е интерполация

Конекторът за Neo4j изгражда текста на Cypher чрез `std::format`, например шаблона `MATCH (n:{}) RETURN {}{}` (`neo4j_live.hpp:261`). Това изглежда като сглобяване на низ и заслужава внимание. Вмъкваните полета обаче са етикетът на възела и проекцията, и двете идват от `constexpr` метаданни на типа, а не от входа. Стойностите по време на изпълнение се предават отделно, като типизирани параметри на Bolt (`neo4j_live.hpp:174`). Форматирането на константи от времето на компилация в шаблон не е интерполация на недоверени данни. Различието е съществено, но то трябва да се демонстрира, а не да се приема. Докъде е стигнала тази демонстрация е разгледано в точка 7.6.

5.3.3. Redis: форматиращият низ не е printf

Конекторът за Redis извиква `redisCommand` с форматиращ низ, в който аргументът е означен с `%s` (`redis_live.hpp:180`). Въпреки сходството с `printf`, `hiredis` не замества текст. Всеки аргумент се кодира като отделен елемент на RESP с префикс за дължина, така че съдържанието му не може да бъде прочетено като протокол. Формата с `argv` (`redis_live.hpp:257`) прави това изрично.

5.3.4. Управление на транзакции

Командите за транзакции изпращат литералните константи `BEGIN`, `COMMIT` и `ROLLBACK` (`postgresql_live.hpp:506, 518, 530`). Те не съдържат потребителски данни и не нарушават гаранцията.

5.4. Проверка на границата

Анализът по хранилища потвърждава предпоставка 2 за всеки конектор поотделно, но оставя един въпрос: стига ли всеки път към конектор само до добре формирано междинно представяне. В хода на настоящата работа беше установено, че отговорът е бил отрицателен (глава VI). Затова в библиотеката е добавена проверка, извиквана от всяка входна точка, която достига конектор.

```
template <typename Query>
constexpr void check_query_shape()
{
    using Q = std::remove_cvref_t<Query>;
    static_assert(is_query_ir<Q>::value,
        "orm: this is not a query. ...");
    if constexpr (is_query_ir<Q>::value)
        static_assert(!query_has_literal<Q>::value,
            "orm: a literal operand reaches execution. ...");
}
```

Listing 2: Проверката на границата (съкратено)

Проверката в листинг 2 има две части. Първата отхвърля всичко, което не е един от четирите вида заявки (`select_query`, `insert_query`, `update_query`, `delete_query`). Анализът на всички конектори показа, че това са единствените видове, които те изпълняват, така че проверката не отхвърля нито една легитимна употреба. Втората част обхожда условията, съединенията и присвояванията в заявката и отхвърля всеки **литерален операнд**: стойност, записана направо в заявката, вместо през заместител. Сравнението с `nullptr` и сравнението между две колони не са литерали и остават разрешени.

Проверката се извиква от седемте входни точки, които достигат конектор: `db::operator<<`, `db::execute`, `db::find_one`, `db::prepare`, `prepared_query::execute` и двете асинхронни съответствия в `async_db`. Пропускането на която и да е от тях би оставило заобиколен път. Затова всяка входна точка се покрива

от отделен компилационен тест (глава VII).

5.5. Известен остатъчен риск

Анализът установи и един път, който е по-слаб от свързването, без да е използван. Функцията `to_sql_literal` (`mysql_live.hpp:249`) изгражда SQL чрез **екраниране** и вмъкване на стойността, вместо чрез свързване. Коментарът ѝ я описва като предназначена за заявки без подготвени изрази. Търсене в каталозите `lib` и `tests` намира само дефиницията и нито едно извикване. Следователно тя е мъртъв код и не засяга текущата гаранция. Тя е отбелязана, защото екранирането е по-слаб механизъм (точка 3.2) и бъдещо използване би отслабило гаранцията за MySQL, без това да е видимо от интерфейса. Препоръката е функцията да бъде премахната или изрично изолирана.

VI. КАЗУС: НЕПРИВЪРЗАНИТЕ ЛИТЕРАЛНИ ОПЕРАНДИ

Глава V установи, че всеки конектор свързва стойностите на заместителите правилно. Тази глава проследява случай, в който предпоставка 2 от точка 3.4 не е била изпълнена, въпреки че всеки конектор поотделно работи коректно. Казусът е избран по две причини. Той демонстрира метода на работата върху реален дефект, от хипотезата през измерването до поправката. Той показва и защо анализът на всеки компонент поотделно не е достатъчен, когато гаранцията зависи от това, какво изобщо може да достигне компонентите.

6.1. Откриване

Отправната точка беше въпросът дали типизираният заместител е *единственият* път, по който стойност достига заявката. Операторите за сравнение на полета са дефинирани чрез макрос в `query/rules.hpp`. Листинг 3 показва съществената му част.

```
template <auto P, typename T>
    requires(!std::is_same_v<T, std::nullptr_t>)
constexpr auto operator==(mem_ptr<P>, T x)
{
    static_assert(comparable_value_types<...>, "...");
    if constexpr (is_field<T>)
        return Rule<decltype(P), "==", decltype(T::get())>(P, T::get());
    else
        return Rule<decltype(P), "==", T>(P, x);   // стойността се записва в заявката
}
```

Listing 3: Операторът за сравнение на поле (съкратено)

От листинга се вижда, че параметърът `T` не е ограничен до заместители. Операторът приема всяка стойност със съвместим тип и я записва в междинното представяне като десен операнд. Следователно съществува и втори път: стойност, записана направо в условието, например `field<&Item::name> == input`, без заместител. Такава стойност се нарича в тази работа **литерален операнд**. Ако тя бъде вмъкната в текста на заявката, гаранцията е нарушена. Ако бъде свързана, гаранцията е запазена. Кодът на оператора не дава отговор, тъй като решението се взема в конектора.

6.2. Хипотеза и измерване

Конекторът за SQLite превежда всеки операнд, който не е колона и не е заместител, в маркер `?` (`sqlite_db.hpp:258`). Литералът не става текст, което изключва инжекцията. Търсене на код, който след това свързва стойността на литерала, не откри такъв в нито един от осемте конектора (седемте работещи и `MockDB`). Хипотезата беше, че маркерът остава несвързан и драйверът го тълкува като `NULL`.

Хипотезата беше проверена с две програми върху SQLite в паметта, с библиотеката във версията преди поправката (`d5795a9`). Таблица 7 показва резултата за заявка за избор.

Таблица 7. Заявка за избор: заместител спрямо литерал

Път	Вход	Редове	Правилно
типизиран заместител	`banana`	1	1
типизиран заместител	`x' OR '1'='1`	0	0
литерал в заявката	`banana`	**0**	1
литерал в заявката	`x' OR '1'='1`	0	0

Резултатът потвърждава хипотезата. Пътят през заместителя връща правилния отговор и при безобиден вход, и при полезен товар. Пътят през литерала връща нула редове дори за `banana`, за който съществува съвпадащ ред. Полезният товар не постига нищо и по двата пътя, но това не означава едно и също: при литерала неуспехът му се дължи на това, че заявката изобщо не работи.

Таблица 8 показва същото при промяна и изтриване.

Таблица 8. Промяна и изтриване: заместител спрямо литерал

Случай	Намерение	Резултат
заместител в `SET`	преименуване на ред 1	редът е преименуван
литерал в `SET`	преименуване на ред 2	**редът е непроменен**
литерал в `WHERE`	изтриване на ред 3	**нищо не е изтрито**

Изходът от двете програми е запазен дословно

в security/evidence/probe_literal_before_fix.txt и security/evidence/probe_update_before_fix.txt.

6.3. Анализ на причината

Случаят с промяната заслужава отделно внимание, защото резултатът му не е просто липса на действие. Аргументите, подадени на execute(), се свързват позиционно, от позиция 1 нататък (sqlite_db.hpp:214). Заявката UPDATE items SET name = ? WHERE id = ? има два маркера, а при изпълнението е подаден един аргумент, предназначен за условието. Маркерът на литерала в SET е първи. Затова аргументът е свързан с него, а маркерът в условието е останал несвързан, тоест NULL, и не е съвпаднал с нито един ред. Това е **объркване на параметрите**: стойност на извикващия код попада на погрешна позиция.

От гледна точка на сигурността дефектът е **затворен отказ**: при грешка системата остава в безопасно състояние. Литералът никога не става текст и не може да промени структурата. Дефектът обаче не е безобиден. Той води до тихо грешни резултати, без грешка и без предупреждение. За приложение, което проверява правомощия чрез заявка, грешен празен резултат може да означава отказ на легитимен достъп. Случай с видим погрешен ефект, например промяна на погрешен ред, не беше наблюдаван в измерените заявки, но не е изключен за всяка форма на заявка.

Остава въпросът защо дефектът не е бил открит от съществуващите тестове. Търсене в тестовете намери литерални операнди само в tests/unit/test_query.cpp, в тестове, които проверяват формата на междинното представяне чрез static_assert и никога не изпълняват заявката. Нито един тест не е изпълнявал заявка с литерален операнд срещу база данни. Дефектът е оцелял в сляпо петно на тестовото покритие, а не поради проектно решение.

6.4. Избор на поправка

Възможни са две поправки.

Свързване на литералите. Всеки конектор събира стойностите на литералите и ги свързва заедно с аргументите на execute(), в общ ред на позициите. Така литералите стават законен втори път за стойности.

Отхвърляне на литералите. Заявка с литерален операнд не се компилира, ако

бъде подадена за изпълнение. Единственият път за стойности остава заместителят.

Изборът падна върху отхвърлянето по три причини. Първо, свързването изисква промяна в превода и свързването на всички седем конектора. Грешка в реда на позициите в който и да е от тях би превърнала затворения отказ в объркване на параметрите с видим ефект, а реализация, която вмъква литерала като текст, би създала истинска инжекция. Отхвърлянето е една проверка на едно място (точка 5.4). Второ, то съответства на тезата на работата: стойностите влизат само през заместители. Трето, поправката не чупи работещ код, защото кодът, който тя отхвърля, и без това е давал грешни резултати.

Цената е, че естествени записи, например условие за положително число, вече изискват заместител. Отхвърлянето засяга и литерали, известни по време на компилация, защото дефектът никога не е зависел от това кога е известна стойността: нито един конектор не свързва литерал.

6.5. Проверка на поправката

Поправката е проверена в два етапа. Преди нея наборът от компилационни тестове беше изпълнен с библиотеката във версия `d5795a9`. Седемте теста за литерални операнди, по един за всяка входна точка и вид заявка, се провалиха с един и същ резултат: програмата се компилира, въпреки че не би трябвало. Резултатът от това изпълнение е запазен в `security/evidence/compile_before_fix.txt`.

След поправката и седемте програми се отхвърлят при компилация по проверяваната причина. Всички тестове преминават: 314-те съществуващи в библиотеката и 23-те нови от набора за оценка. Последното е съществено: то показва, че поправката затваря дефекта, без да засегне работещ код. Подробните резултати са в глава VII.

6.6. Изводи от казуса

Казусът води до два извода, които надхвърлят конкретния дефект. Първият е, че гаранцията за сигурност на една абстракция не е сума от гаранциите на компонентите ѝ. Всеки конектор свързва заместителите правилно, но никой от тях не е отговарял за въпроса какво изобщо може да ги достигне. Вторият е, че тест, който проверява само формата на заявката, не проверява поведението ѝ. Дефектът оцеля, защото литералите бяха тествани като структура, но не и като изпълнени заявки. Методът на настоящата работа, който изисква всяко твърдение да бъде измерено, а не изведено, е причината дефектът да бъде

открит.

VII. ОЦЕНКА

Главите V и VI описаха как гаранцията се постига и как беше затворен един пропуск в нея. Настоящата глава проверява гаранцията по метода от точка 4.6 и съпоставя резултатите с критериите от таблица 4. Наблюденията се представят първо, а тълкуването им следва отделно, за да може читателят да прецени едното независимо от другото.

7.1. Среда и възпроизводимост

Измерванията със SQLite и компилационните тестове са извършени с изходния код във версия `a17f4e7`, при следната среда:

- операционна система Ubuntu 24.04.4 LTS под WSL2, ядро `6.6.114.1-microsoft-standard-WSL2`;
- компилатори GCC 14.2.0 и Clang 18.1.3, стандарт C++23;
- SQLite 3.45.1, база данни в паметта, нова за всяко изпълнение;
- CMake 3.28.3.

Наборът се изгражда и изпълнява с опцията `ORM_ENABLE_SECURITY`, както е описано в приложение А. Всеки резултат се записва в каталога `security/evidence/` в изходния код, а не в каталога за изграждане. Така всяко изпълнение е преглеждаема разлика в системата за контрол на версиите и всяко число в тази глава може да бъде проследено до файл. От версия `a17f4e7` нататък наборът се изпълнява и при всяка промяна в системата за непрекъсната интеграция, като изпълнението за самата версия `a17f4e7` е успешно.

Измерванията със сървърните хранилища са извършени в друга среда, защото изискват работещи сървъри. Те се изпълняват от работния процес `Docker Integration Tests` на системата за непрекъсната интеграция на GitHub, а резултатите са записани в същия каталог `security/evidence/`. Средата е:

- изпълнител `ubuntu-24.04`, версия на образа на изпълнителя `20261025.353.1`;
- образ `test-all`, построен от `docker/Dockerfile` върху Ubuntu 24.04, с GCC 13.3.0 и стандарт C++23, с включена опция `ORM_ENABLE_SECURITY`;

- услуги от `docker/docker-compose.yml`, стартирани с Docker Compose: `postgres:15` (сървър 15.19), `mariadb:10.11` (сървър 10.11.19), `mongo:7.0` (сървър 7.0.45), `redis:7.2-alpine` (сървър 7.2.17) и `cassandra:4.1` (сървър 4.1.12). Версиите на сървърите са прочетени от самите сървъри и са записани в първите редове на всеки файл `differential_*_summary.txt`;
- изпълнение на работния процес с номер `41868248172`, чиито файлове са качени като артефакт `security-evidence`. Изпълнението от предишната версия на програмите (номер `41865383839`) дава идентични файлове `.csv`, което е проверка на повторяемостта.

Сървърът, наречен тук MySQL, е MariaDB. Конекторът на библиотеката е построен върху MariaDB Connector/C, който се нуждае от неблокиращия интерфейс `mysql_*_start`, и затова композиращият файл използва MariaDB.

7.2. Корпус от полезни товари

Корпусът съдържа 66 полезни товара от 11 публични източника. Всеки ред съдържа препратка към източника си и нито един полезен товар не е съставен за целите на работата. Таблица 9 показва разпределението.

Таблица 9. Състав на корпуса

Клас	Брой	Диалекти
SQL	36	общи 16, MySQL 8, PostgreSQL 6, SQLite 6
NoSQL (MongoDB)	15	инжекция на оператори
Cypher (Neo4j)	6	
CQL (Cassandra)	4	
Redis	5	инжекция на протоколно ниво
Общо	66	

Разпределението е неравномерно и това е следствие от правилото полезен товар да се включва само ако е открит дословно в източник. Три полезни товара са публикувани

от източника с незапълнено поле, например `[RANDNUM]`. Те се изпълняват точно както са публикувани, защото запълването на полето би означавало съставяне на нов текст. Корпусът изрично изброява и пропуските си, сред които липсата на проверен полезен товар с `UNION` и с натрупани заявки за SQLite, както и с `UNION` за Cypher.

7.3. Отрицателни компилационни тестове

Наборът съдържа 18 компилационни теста: 15 програми, които трябва да бъдат отхвърлени, и 3 контролни случая, които трябва да се компилират. Таблица 10 показва резултата при двата компилатора.

Таблица 10. Резултати от компилационните тестове

Случай	Ниво	Очакване	GCC	Clang
00 документирана заявка	контрола	компилира	да	да
14 сравнение с `nullptr`	контрола	компилира	да	да
15 сравнение на две колони	контрола	компилира	да	да
06 низ към целочислен заместител	1	отхвърля	да	да
01 име на колона по време на изпълнение	2	отхвърля	да	да
16 име на колона в `std::string`	2	отхвърля	да	да
02 име на таблица по време на изпълнение	2	отхвърля	да	да
04 колона, избрана по време на изпълнение	2	отхвърля	да	да
17 две колони в една променлива	2	отхвърля	да	да
03 текст на заявка към `execute`	3	отхвърля	да	да
05 условие като текст	3	отхвърля	да	да
07 литерал, `execute`	литерал	отхвърля	да	да
08 литерал, `operator<<`	литерал	отхвърля	да	да
09 литерал, `find_one`	литерал	отхвърля	да	да
10 литерал, `prepare`	литерал	отхвърля	да	да
11 литерал в `SET`	литерал	отхвърля	да	да
12 литерал в `DELETE`	литерал	отхвърля	да	да
13 литерал, константа	литерал	отхвърля	да	да

„Да“ в таблицата означава, че програмата е дала очаквания резултат и, при отхвърляне, че диагностиката съответства на образеца за проверяваната причина. Диагностиките са запазени поотделно за всеки компилатор в `security/evidence/compile/`.

Два от тестовете бяха уточнени след първото изпълнение и това има значение за тълкуването. Първоначалните варианти на случаи 01 и 04 бяха отхвърлени, но по причина, различна от проверяваната, и затова се отчетоха като неуспешни. Случай 01 опитваше да използва `std::string` като име на колона и беше отхвърлен поради несъответствие на типа, преди въпросът за константността изобщо да възникне. Той беше разделен на случай 01, който подава стойност с правилния тип, но известна едва по време на изпълнение, и случай 16, който запазва варианта с `std::string`. Случай 04 опитваше да избере между две колони и беше отхвърлен със съобщението, че двата указателя към член имат различни типове. Това разкри по-силно свойство от проверяваното: понеже името на колоната е част от типа на свойството, две колони с един и същ тип на стойността (тук и двете `int`) са различни типове, и една променлива не може да обхваща и двете. То е отделено в случай 17, а случай 04 проверява само константността. Записът от първото изпълнение е запазен в `security/evidence/compile_before_fix.txt`.

7.4. Диференциален тест със SQLite

От корпуса към SQLite са приложими 22 полезни товара: общите 16 и специфичните за SQLite 6. Таблица 11 обобщава резултата.

Таблица 11. Диференциален тест със SQLite, 22 полезни товара

Резултат	Наивна заявка	Библиотеката
Компрометирана: върнати редове	4	0
Компрометирана: промяна на състоянието	0	0
Компрометирана: принудително изпълнение	0	0
Синтактично нарушена	10	0
Без наблюдаем ефект	8	22

Таблица 12 показва резултата на наивната заявка по техники.

Таблица 12. Резултат на наивната заявка по техники

Техника	Брой	Резултат
тавтология	2	и двата компрометират (3 и 1 върнати реда)
`UNION`	2	и двата компрометират (по 1 върнат ред)
сляпа атака по време	6	и шестте нарушават синтаксиса
натрупана заявка	1	нарушава синтаксиса
сляпа атака по истинност	3	2 без ефект, 1 нарушава синтаксиса
коментар	3	2 без ефект, 1 нарушава синтаксиса
изброяване	2	1 без ефект, 1 нарушава синтаксиса
предизвикване на грешка	1	без ефект
заобикаляне на екраниране	2	без ефект

Наблюденията са следните. Наивната заявка е компрометирана от 4 от 22 полезни товара. И четирите са върнали редове, които заявката не би трябвало да върне. Единият от двата полезни товара с `UNION`, `SQL-003`, е върнал реда с потребителското име и паролата от таблицата `users`, тоест е извлякъл съхранена идентификационна информация. Другият, `SQL-004`, е върнал ред с избрани от нападателя стойности, техника за заобикаляне на удостоверяване. През библиотеката и 22-та полезни товара не са имали наблюдаем ефект. Пълният резултат по полезни товари е в `security/evidence/differential_sqlite.csv`.

7.5. Тълкуване

Резултатът на библиотеката, 22 от 22 без ефект, придобива смисъл едва на фона на резултата на наивната заявка. Четирите компрометирания показват, че корпусът и постановката са способни да разкрият уязвимост, тоест че нулата при библиотеката не се дължи на безсилен корпус.

Десетте синтактично нарушени заявки заслужават отделен коментар. Те не са неуспех на атаката в смисъла на защитата. Полезният товар е достигнал граматиката на SQLite и е нарушил заявката, но не е постигнал ефект. Причината е проверена за всеки случай. Пет от шестте слепи атаки по време използват конструкции на други системи, които SQLite не поддържа: `SLEEP` на MySQL, `WAITFOR DELAY` на SQL Server и `dbms_pipe` на Oracle. Шестата, `SQL-034`, е специфична за SQLite, но е публикувана с незапълнени полета и затова не е синтактично валидна. Натрупаната заявка `SQL-017` извиква процедурата `xp_cmdshell` на SQL Server. Срещу библиотеката същите полезни товари не достигат граматиката изобщо: те пристигат като стойност на свързан параметър и се сравняват като низ.

Осемте полезни товара без ефект при наивната заявка са предимно слепи атаки и опити за заобикаляне на екраниране. Слепата атака по дефиниция не дава видим резултат при едно изпълнение: тя разчита на разлика между две изпълнения. Затова оракулът, който вижда само ефекти, не я отчита. Както е посочено в точка 4.7, броят на компрометиранията на наивната заявка е долна граница.

7.6. Сървърните хранилища

Диференциалният тест беше разширен до пет работещи сървъра: PostgreSQL, MySQL, MongoDB, Redis и Cassandra. Всеки има своя програма, например

runtime/differential_postgresql.cpp, върху общ драйвер, който чете същия корпус, записва същите колони и прилага същия оракул като теста със SQLite. Измерванията са извършени в системата за непрекъсната интеграция върху услугите от файла docker/docker-compose.yml, както е описано в точка 7.1.

Наивният път е реалистичната уязвима конструкция за всяко хранилище.

- **PostgreSQL и MySQL:** SQL текст, сглобен чрез конкатенация. При PostgreSQL той се изпраща с простия протокол за заявки (PQsendQuery), а при MySQL по връзка с флага CLIENT_MULTI_STATEMENTS. И в двата случая клиентът допуска натрупани заявки, което е най-лошият реалистичен случай.
- **Cassandra:** CQL текст, сглобен чрез конкатенация и изпратен като неподготвена заявка.
- **MongoDB:** филтър, сглобен като JSON текст и анализиран с bson_new_from_json, или израз $where. Корпусът смесва полезни товари за различни контексти, затова се използват пет конструкции, а се запазва резултатът с най-тежък ефект.
- **Redis:** команда, сглобена като текст и изпратена в инлайн протокола, така че CR LF в полезния товар започва втора команда. Това не е форматирането %s на hiredis, което е безопасно.

Библиотеката получава полезния товар като типизиран заместител през db.execute. Заявката е търсене по име в малка предварително попълнена таблица, колекция, пространство от ключове или набор от ключове. Втора таблица или ключ съдържа тайна, за да има какво да достигне UNION или допълнителна команда. Всеки полезен товар започва от ново състояние. Преди и след него се снема снимка на данните и схемата, а при MySQL, PostgreSQL и Redis и на акаунтите или конфигурацията. Принудителното изпълнение се отчита по времеви праг от 2500 ms. Преди полезните товари всяка програма проверява контролно търсене на съществуваща стойност, което трябва да върне точно един ред по двата пътя.

Правилото за неуспех е различно от това при SQLite. Програмата за сървър завършва с грешка само ако някой полезен товар има ефект през библиотеката. Ако наивният път никога не е компрометиран, тя не се проваля, а записва в резюмето си, че сравнението за това хранилище е неубедително.

Таблица 13 обобщава резултатите.

Таблица 13. Диференциален тест със сървърните хранилища

Хранилище (сървър)	Прил.	Наивна: компром.	Наивна: синт. нарушена	Наивна: без ефект	Библиотека: без ефект	Заключ.
PostgreSQL 15.19	22	2	12	8	22	да
MySQL (MariaDB 10.11.19)	24	2	12	10	24	да
MongoDB 7.0.45	15	2	13	0	15	да
Redis 7.2.17	5	2	3	0	5	да
Cassandra 4.1.12	4	0	3	1	4	не
Neo4j	6	не е измерено				не

В последната колона *да* означава, че сравнението е заключително: наивният път е компрометиран поне веднъж. Числата са от файловете `differential_postgresql_summary.txt`, `differential_mysql_summary.txt`, `differential_mongodb_summary.txt`, `differential_redis_summary.txt` и `differential_cassandra_summary.txt` в каталога `security/evidence/`. Резултатът за всеки полезен товар е в съответните файлове `.csv`.

Наблюденията по хранилища са следните.

PostgreSQL. Наивната заявка е компрометирана от два полезни товара, двата с върнати редове: тавтологиите `SQL-001` (3 реда) и `SQL-002` (1 ред). Двата полезни товара с `UNION` са нарушили синтаксиса. През библиотеката 22 от 22 са без ефект.

MySQL. Наивната заявка е компрометирана от два полезни товара: `SQL-024` е върнал 3 реда, а `SQL-006` е надхвърлил времевия праг, тоест е принудил сървъра да изпълнява. Тавтологиите `SQL-001` и `SQL-002` са нарушили синтаксиса, както и двата полезни товара с `UNION`. През библиотеката 24 от 24 са без ефект.

MongoDB. Наивният път е компрометиран от два полезни товара. `NOSQL-007` е върнал и трите документа, когато текстът на полезния товар е бил целият филтър, а `NOSQL-014`, цикълът за заетост в `$where`, е надхвърлил времевия праг и е бил прекъснат от ограничението `maxTimeMS`. Най-тежкият ефект на останалите 13 е нарушаване на синтаксиса при поне една конструкция, но не и компрометиране. През библиотеката 15

от 15 са без ефект. Документите в колекцията съдържат само полетата `id` и `name`, затова полезните товари, писани за схема с полета `username` и `password`, съвпадат само тогава, когато операторът е удовлетворен от липсата на полето.

Redis. Наивният път е компрометиран от два полезни товара, `REDIS-001` и `REDIS-002`. И двата са изпълнили вмъкнатата команда `FLUSHALL`: снимката на ключовете след изпълнението се различава от снимката преди него. Останалите три полезни товара фалшифицират отговори на сървъра, тоест са насочени към клиента, а не към сървъра. При наивния път те са отчетени като синтактично нарушени, защото сървърът е приел вмъкнатите редове за команди и ги е отхвърлил. През библиотеката 5 от 5 са без ефект. След всеки полезен товар връзката на библиотеката получава `PING` и отговорът е бил очакваният, тоест потокът от отговори е останал синхронизиран.

Cassandra. Наивният път не е компрометиран от нито един от четирите полезни товара: три са нарушили синтаксиса, а един, коментарът `CQL-001`, е останал без ефект. През библиотеката 4 от 4 са без ефект.

Neo4j. Не е измерен. Образът `test-all` не съдържа клиентската библиотека `libneo4j-client`, затова конекторът `orm::neo4j_live` не се изгражда, а файлът за композиране няма услуга Neo4j. Измерването му изисква нов етап в образа и нова услуга. Причината е записана в `security/evidence/differential_neo4j_summary.txt`.

Общо през библиотеката 70 от 70 приложими полезни товара за петте сървъра са без ефект, а заедно със SQLite резултатът е 92 от 92. Наивният път е компрометиран от 8 от 70 полезни товара за сървърите, по два при всяко от четирите хранилища, за които сравнението е заключително.

Тълкуването е следното. При PostgreSQL, MySQL, MongoDB и Redis сравнението е заключително: наивният път е компрометиран поне веднъж, а библиотеката не е. Това е същата структура на доказателство като при SQLite. Компрометиранията са от три вида, а не повторение на един: върнати редове при PostgreSQL, MySQL и MongoDB, принудително изпълнение при MySQL и MongoDB и промяна на състоянието при Redis. Всяко от четирите хранилища е компрометирано от по два полезни товара, тоест от по-малко от четирите при SQLite. Причината не е проверявана полезен товар по полезен товар. Корпусът е разделен по диалекти, а постановката има една форма на заявка за хранилище. Затова тези числа са долна граница, както при SQLite.

При Cassandra сравнението е незаключително. Нулата при библиотеката не доказва

нищо, защото в тази постановка нищо не показва, че корпусът може да разкрие уязвимост. Възможно обяснение е в езика: CQL приема една инструкция на заявка и няма `UNION`. Това е хипотеза, а не измерване. За Cassandra гаранцията остава подкрепена от анализа на изходния код в точка 5.3 и от резултата 4 от 4 без ефект, но не и от диференциално доказателство.

При Redis три от петте полезни товара са насочени към клиента, а не към сървъра. Заключителността на Redis се дължи на двата полезни товара, които вмъкват команда. При MongoDB резултатът е долна граница и по още една причина: колекцията има само полетата `id` и `name`, а наивните конструкции са пет, избрани от автора. Друга конструкция би могла да промени наивните числа. Тя не променя резултата на библиотеката, защото според анализа в точка 5.3 стойността достига сървъра като поле на документ, а не като текст за анализ.

7.7. Съпоставка с критериите за успех

Таблица 14 съпоставя резултатите с критериите от таблица 4.

Таблица 14. Съпоставка с критериите за успех

№	Оценка	Основание
К1	изпълнен за шест от седемте хранилища	92 от 92 полезни товара без ефект през библиотеката: SQLite 22, PostgreSQL 22, MySQL 24, MongoDB 15, Redis 5, Cassandra 4; Neo4j не е измерен
К2	изпълнен	15 от 15 отхвърляния по проверяваната причина и 3 от 3 контролни случая, при GCC и при Clang
К3	изпълнен за пет от седемте хранилища	наивният път е компрометиран от 4 полезни товара при SQLite и от по 2 при PostgreSQL, MySQL, MongoDB и Redis; при Cassandra от нито един, а Neo4j не е измерен
К4	изпълнен	назовани граници (точка 4.3), демонстриран и затворен пропуск (глава VI), отбелязан остатъчен риск (точка 5.5)

Критериите К2 и К4 са изпълнени изцяло. Критерият К1 е изпълнен за шест от

седемте хранилища. Критерият К3 е изпълнен за пет: при Cassandra наивният път не е компрометиран и затова нулата при библиотеката не доказва нищо, а Neo4j не е измерен. За тези две хранилища гаранцията се подкрепя само от анализа на изходния код. Разликата между двата вида подкрепа е съществена и не се замъглява: анализът показва механизма, но само измерването може да покаже, че механизмът работи така, както е описан.

7.8. Заплахи за валидността

Вътрешна валидност. Оракулът вижда само ефекти и подценява наивния път. Това изместване работи срещу тезата, а не в нейна полза. Компилационните тестове използват конектора `MockDB`, който превежда заявките без база данни. Проверката на границата обаче се извършва в общия слой, преди конектора, така че изборът на конектор не влияе на резултата от тези тестове.

Конструктивна валидност. Диференциалният тест използва една форма на заявка с две колони. Тя е избрана, за да дава шанс на най-разпространените полезни товари с `UNION`. Друга форма би могла да направи наивния път уязвим за други полезни товари. Това не засяга резултата на библиотеката, защото при нея стойността не достига граматиката при никоя форма. Наивните конструкции за сървърните хранилища са избрани от автора на работата, като реалистични за съответното хранилище, а не заимствани от измерена система. При MongoDB те са пет и се запазва най-тежкият резултат, което повишава броя на наивните компрометирания. Това изместване не облагодетелства библиотеката.

Външна валидност. Измерването е извършено върху шест от седемте хранилища, по една версия на сървъра за всяко, и върху един корпус от 66 полезни товара. Neo4j не е измерен. Корпусът не е изчерпателен и съдържа пропуски, които са изброени в самия него. Твърдението, че гаранцията се отнася за всички входове, не се основава на корпуса, а на аргумента от точка 3.4, който компилационните тестове проверяват. Корпусът показва, че аргументът издържа на реални атаки, но не замества аргумента.

Надеждност. Резултатите са детерминирани. Измерванията на времето са изключени от записаните резултати, за да не предизвикват разлики между изпълненията. При сървърните хранилища принудителното изпълнение се отчита по времеви праг от 2500 ms, така че тези резултати зависят от времето в средата. Два изпълнения в системата за непрекъсната интеграция дадоха идентични файлове `.csv`, но това не е доказателство за независимост от натоварването на изпълнителя. Всеки резултат е записан във файл в

изходния код и може да бъде получен повторно с командите от приложение А.

ЗАКЛЮЧЕНИЕ

Работата постави въпроса дали една абстракция, конструираща заявките по време на компилация, може да елиминира цяло семейство от класове инжекции при хетерогенни хранилища за данни, и къде спира тази гаранция.

Отговор на изследователския въпрос

Отговорът е положителен, с ясно назовани граници. Когато имената на таблици и колони и формата на заявката са аргументи на шаблони, а стойностите достигат заявката само чрез свързани типизирани заместители, недоверен вход не може да промени структурата на заявката. Това следва от определението на инжекцията като промяна на синтактичната структура [4] и от изискването аргументите на шаблон да бъдат константни изрази. Аргументът е проверен, а не само изведен: 15 от 15 опита идентификатор или структура да зависят от стойност по време на изпълнение са отхвърлени от компилатора по проверяваната причина, при два компилатора, а при шестте измерени хранилища нито един от 92 приложими полезни товара не е имал ефект през библиотеката. Наивният път е компрометиран от 4 от 22 при SQLite и от по 2 при PostgreSQL, MySQL, MongoDB и Redis. При Cassandra той не е компрометиран, затова там сравнението е неубедително.

Гаранцията обаче не беше пълна в началото на изследването. Стойност, записана направо в заявката, се приемаше и оставаше несвързана. Това не позволяваше инжекция, но водеше до тихо грешни резултати. Пропускът е затворен. Именно този резултат показва най-ясно защо втората част на изследователския въпрос е равностойна на първата: гаранцията беше изказана като пълна, преди да бъде измерена.

Приноси

1. **Научно-приложен принос.** Аргумент, който свързва формалното определение на инжекцията с изискването за константни изрази в C++ и го разширява от SQL към пет класа инжекции при седем хранилища. Установено е, че гаранцията се постига с четири различни механизма и че при две от хранилищата липсва текстова граматика, от която полезен товар би могъл да излезе.

2. **Приложен принос.** Възпроизводим набор за оценка, включен в системата за непрекъсната интеграция на библиотеката: 18 компилационни теста с контролни

случаи и проверка на диагностиката, корпус от 66 полезни товара с посочен източник за всеки и диференциален тест със съзнателно уязвима база за сравнение.

3. **Приложен принос.** Открит, измерен и затворен пропуск в гаранцията на библиотеката, както и проверка на границата на интерфейса, която отхвърля текст на заявка със съобщение, назоваващо причината.

4. **Отрицателни резултати.** Назовани граници на гаранцията и остатъчен риск в неизползван път с по-слаб механизъм, който е мъртъв код, но би отслабил гаранцията, ако бъде използван.

Ограничения

Основното ограничение е, че диференциалният тест не обхваща всички хранилища. Neo4j не е измерен, а конекторът му не се изпълнява дори от съществуващите интеграционни тестове на библиотеката. При Cassandra наивният път не е компрометиран от нито един от четирите полезни товара, така че сравнението е неубедително и гаранцията там се подкрепя само от анализа на изходния код. Наивните конструкции за сървърните хранилища са избрани от автора, а при MongoDB са пет с най-тежък резултат. Оракулът вижда само ефекти и не отчита слепите атаки. Тестът използва една форма на заявка за хранилище, а корпусът не е изчерпателен. Най-сетне, самата гаранция има цена: схеми, определяни по време на изпълнение, не се поддържат, защото именно невъзможността идентификатор да зависи от входа е същността на защитата.

Бъдеща работа

Насоките произтичат пряко от ограниченията. Първата е добавянето на Neo4j към образа за контейнери и към файла за композиране, за да бъде измерен и той. Втората е заключително сравнение за Cassandra: форма на заявка или оракул, при които наивният път може да бъде компрометиран, ако това е възможно изобщо. Третата е оракул по двойки, който сравнява две изпълнения и така отчита слепите атаки. Четвъртата е премахването на мъртвия път с екраниране. Петата е алтернативният дизайн, при който литералите се свързват вместо да се отхвърлят, оценен по същия метод. Шестата е самостоятелен анализ на операциите по схемата и на инжекцията от втори ред, които останаха извън обхвата.

ИЗПОЛЗВАНИ ИЗТОЧНИЦИ

[1] OWASP Foundation, “OWASP Top 10:2025.” https://top10.owasp.org/2025/, 2025. Достъп: 4 ноември 2026.

[2] OWASP Foundation, “SQL Injection Prevention Cheat Sheet.” https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html. Достъп: 4 ноември 2026.

[3] Cybersecurity and Infrastructure Security Agency and Federal Bureau of Investigation, “Secure by Design Alert: Eliminating SQL Injection Vulnerabilities in Software.” https://www.cisa.gov/resources-tools/resources/secure-design-alert-eliminating-sql-injection-vulnerabilities-software, 2024. Публикувано на 25 март 2024. Прочетен е пълният текст във файла на страницата (PDF). Достъп: 4 ноември 2026.

[4] Z. Su and G. Wassermann, “The Essence of Command Injection Attacks in Web Applications,” in *Conference Record of the 33rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL ’06)*, (Charleston, South Carolina, USA), pp. 372–382, ACM, 2006.

[5] Алекс Цветанов, “Разработване на библиотека за обектно-релационно картографиране по време на компилация.” Дипломна работа за ОКС „Бакалавър“, Технически университет, София, Факултет „Компютърни системи и технологии“, 2026. Изходен код: https://github.com/cpp-for-everything/ORM.

[6] OWASP Foundation, “A05:2025 Injection.” https://top10.owasp.org/2025/A05_2025-Injection/, 2025. Достъп: 4 ноември 2026.

[7] MITRE Corporation, “CWE-89: Improper Neutralization of Special Elements used in an SQL Command (SQL Injection).” https://cwe.mitre.org/data/definitions/89.html. Версия 5.0. Достъп: 4 ноември 2026.

[8] MITRE Corporation, “CWE-943: Improper Neutralization of Special Elements in Data Query Logic.” https://cwe.mitre.org/data/definitions/943.html. Версия 5.0. Достъп: 4 ноември 2026.

[9] MITRE Corporation, “2025 CWE Top 25 Most Dangerous Software Weaknesses.” https://cwe.mitre.org/top25/archive/2025/2025_cwe_top25.html, 2025. Достъп: 4 ноември 2026.

[10] National Institute of Standards and Technology, “CVE-2023-34362 Detail.” https://nvd.nist.gov/vuln/detail/CVE-2023-34362. Данните са прочетени през програмния интерфейс на NVD, https://services.nvd.nist.gov/rest/json/cves/2.0?cveId=CVE-2023-34362. Достъп: 4 ноември 2026.

[11] Federal Bureau of Investigation and Cybersecurity and Infrastructure Security Agency, “#StopRansomware: CL0P Ransomware Gang Exploits CVE-2023-34362 MOVEit Vulnerability, Alert AA23-158A.” https://www.cisa.gov/news-events/cybersecurity-advisories/aa23-158a, 2023. Публикувано на 7 юни 2023, обновено на 16 юни 2023. Достъп: 4 ноември 2026.

[12] U.S. Department of Justice, U.S. Attorney’s Office, District of New Jersey, “Three Men Indicted for Hacking into Five Corporate Entities, including Heartland, 7-Eleven, and Hannaford, With Over 130 Million Credit and Debit Card Numbers Stolen.” https://www.justice.gov/sites/default/files/usao-nj/legacy/2013/11/29/gonz0817%20rel.pdf, 2009. Съобщение за печата от 17 август 2009. Достъп: 4 ноември 2026.

[13] U.S. Department of Justice, Criminal Division, “Leader of Hacking Ring Sentenced for Massive Identity Thefts from Payment Processor and U.S. Retail Networks.” https://justice.gov/archive/usao/nj/Press/files/pdffiles/2010/dojgonzalez0326rel.pdf, 2010. Съобщение за печата от 26 март 2010. Достъп: 4 ноември 2026.

[14] Information Commissioner’s Office, “TalkTalk gets record £400,000 fine for failing to prevent October 2015 attack.” https://web.archive.org/web/2017/https://ico.org.uk/about-the-ico/news-and-events/news-and-blogs/2016/10/talktalk-gets-record-400-000-fine-for-failing-to-prevent-october-2015-attack/, 2016. Съобщение от 5 октомври 2016, прочетено в архивно копие, защото сайтът на ICO отказва автоматизиран достъп. Достъп: 4 ноември 2026.

[15] Council of Europe, “Convention on Cybercrime, ETS No. 185.” https://rm.coe.int/16800 81561, 2001. Будапеща, 23 ноември 2001. Прочетен е английският текст в архивно копие. Достъп: 4 ноември 2026.

[16] Европейски парламент и Съвет на Европейския съюз, “Директива 2013/40/ЕС на Европейския парламент и на Съвета от 12 август 2013 г. относно атаките срещу информационните системи и за замяна на Рамково решение 2005/222/ПВР на Съвета.” https://eur-lex.europa.eu/legal-content/BG/TXT/?uri=CELEX:32013L0040, 2013. Официален вестник на ЕС, L 218, с. 8. Достъп: 4 ноември 2026.

[17] Народно събрание на Република България, “Наказателен кодекс.” https://www.lex.bg/laws/ldoc/1589654529, 1968. Обн. ДВ, бр. 26 от 2 април 1968 г. Прочетен е текстът на lex.bg към 4 ноември 2026 (чл. 319а до 319е); последната отбелязана в него промяна на кодекса е ДВ, бр. 55 от 16 юни 2026 г. Достъп: 4 ноември 2026.

[18] Европейски парламент и Съвет на Европейския съюз, “Регламент (ЕС) 2016/679 на Европейския парламент и на Съвета от 27 април 2016 г. относно защитата на физическите лица във връзка с обработването на лични данни и относно свободното движение на такива данни (Общ регламент относно защитата на данните).” https://eur-lex.europa.eu/legal-content/BG/TXT/?uri=CELEX:32016R0679, 2016. Официален вестник на ЕС, L 119, с. 1. Достъп: 4 ноември 2026.

[19] Европейски парламент и Съвет на Европейския съюз, “Директива (ЕС) 2022/2555 на Европейския парламент и на Съвета от 14 декември 2022 г. относно мерки за високо общо ниво на киберсигурност в Съюза (Директива МИС 2).” https://eur-lex.europa.eu/legal-content/BG/TXT/?uri=CELEX:32022L2555, 2022. Официален вестник на ЕС, L 333, с. 80. Достъп: 4 ноември 2026.

[20] CMS, “Bulgaria adopts NIS2-aligned cybersecurity law.” https://cms.law/en/bgr/legal-updates/bulgaria-adopts-nis2-aligned-cybersecurity-law, 2026. Вторичен източник (правна кантора). Достъп: 4 ноември 2026.

[21] RISI, “Bulgaria, NIS2 transposition tracker.” https://www.risidata.com/regulations/nis2/countries/bg, 2026. Вторичен източник. Достъп: 4 ноември 2026.

[22] Комисия за регулиране на съобщенията, “Закон за киберсигурност (пречистен текст).” https://crc.bg/files/2026%20%D0%9F%D0%A0%D0%90%D0%92%D0%9D%D0%90/%D0%97%D0%90%D0%9A%D0%9E%D0%9D%20%D0%97%D0%90%20%D0%9A%D0%98%D0%91%D0%95%D0%A0%D0%A1%D0%98%D0%93%D0%A3%D0%A0%D0%9D%D0%9E%D0%A1%D0%A2.pdf, 2026. От заглавната бележка на

файла е използвано само обнародването: ДВ, бр. 94 от 2018 г. и промяна в ДВ, бр. 17 от 13-о число на 2026 г. (месецът в бележката не е четим при извличане на текста). Текстът на членовете не е използван. Достъп: 4 ноември 2026.

[23] Neo4j, Inc., “Protecting against Cypher Injection.” https://support.neo4j.com/s/article/4407911287443-Protecting-against-Cypher-Injection/. Статия от базата знания, дата на статията 10 юли 2025. Достъп: 4 ноември 2026.

[24] The Apache Software Foundation, “Cassandra Query Language: Definitions, Prepared Statements (documentation 3.11).” https://cassandra.apache.org/doc/3.11/cassandra/cql/definitions.html. Достъп: 4 ноември 2026.

[25] Microsoft, “SQL Queries, EF Core.” https://learn.microsoft.com/en-us/ef/core/querying/sql-queries. Microsoft Learn. Достъп: 4 ноември 2026.

[26] W. G. J. Halfond, J. Viegas, and A. Orso, “A Classification of SQL Injection Attacks and Countermeasures,” in *Proceedings of the IEEE International Symposium on Secure Software Engineering (ISSSE)*, IEEE, 2006. Прочетен е текстът от страницата на автора (архивно копие). Достъп: 4 ноември 2026.

[27] OWASP ModSecurity project, “ModSecurity: open source, cross platform web application firewall engine.” https://github.com/owasp-modsecurity/ModSecurity. Достъп: 4 ноември 2026.

[28] OWASP CRS project, “Anomaly Scoring, OWASP CRS documentation.” https://coreruleset.org/docs/2-how-crs-works/2-1-anomaly_scoring/. Достъп: 4 ноември 2026.

[29] OWASP CRS project, “Paranoia Levels, OWASP CRS documentation.” https://coreruleset.org/docs/2-how-crs-works/2-2-paranoia_levels/. Достъп: 4 ноември 2026.

[30] W. G. J. Halfond and A. Orso, “AMNESIA: Analysis and Monitoring for NEutralizing SQL-Injection Attacks,” in *Proceedings of the 20th IEEE/ACM International Conference on Automated Software Engineering (ASE ’05)*, pp. 174–183, ACM, 2005.

[31] S. Bandhakavi, P. Bisht, P. Madhusudan, and V. N. Venkatakrishnan, “CANDID: Preventing SQL Injection Attacks Using Dynamic Candidate Evaluations,” in *Proceedings of the 14th ACM Conference on Computer and Communications Security (CCS ’07)*, pp. 12–24, ACM, 2007.

[32] S. W. Boyd and A. D. Keromytis, “SQLrand: Preventing SQL Injection Attacks,” in *Applied Cryptography and Network Security (ACNS 2004)*, Lecture Notes in Computer Science, pp. 292–302, Springer, 2004.

[33] A. Ron, A. Shulman-Peleg, and E. Bronshtein, “No SQL, No Injection? Examining NoSQL Security.” https://arxiv.org/abs/1506.04082, 2015. Предпечат, arXiv:1506.04082, отбелязан в него като статия за семинара W2SP 2015. Достъп: 4 ноември 2026.

[34] A. Ron, A. Shulman-Peleg, and A. Puzanov, “Analysis and Mitigation of NoSQL Injections,” *IEEE Security & Privacy*, vol. 14, no. 2, pp. 30–39, 2016.

[35] OWASP Foundation, “Testing for NoSQL Injection, OWASP Web Security Testing Guide v4.2.” https://wstg.owasp.org/v4.2/4-Web_Application_Security_Testing/07-Input_Validation_Testing/05.6-Testing_for_NoSQL_Injection/. Достъп: 4 ноември 2026.

[36] Neo4j, Inc., “Cypher Manual: Parameters.” https://neo4j.com/docs/cypher-manual/current/syntax/parameters/. Достъп: 4 ноември 2026.

[37] Redis Ltd., “Redis security.” https://redis.io/docs/latest/operate/oss_and_stack/management/security/. Достъп: 4 ноември 2026.

[38] Data Geekery GmbH, “SQL injection, jOOQ manual.” https://www.jooq.org/doc/latest/manual/sql-building/bind-values/sql-injection/. Достъп: 4 ноември 2026.

[39] Data Geekery GmbH, “API validation using the Checker Framework or Error Prone, jOOQ manual.” https://www.jooq.org/doc/latest/manual/sql-building/checker-framework/. Достъп: 4 ноември 2026.

[40] Diesel project, “Diesel: a safe, extensible ORM and query builder for Rust.” https://diesel.rs/. Достъп: 4 ноември 2026.

[41] Diesel project, “Crate `diesel`, documentation 2.2.x.” https://docs.diesel.rs/2.2.x/diesel/index.html. Достъп: 4 ноември 2026.

[42] Diesel project, “Function `diesel::sql_query`.” https://docs.diesel.rs/master/diesel/fn.sql_query.html. Документация на разклонението master. Достъп: 4 ноември 2026.

[43] SQLx contributors, “SQLx: the Rust SQL toolkit.” https://github.com/transact-rs/sqlx. Достъп: 4 ноември 2026.

[44] SQLx contributors, “Function `sqlx::query`, version 0.9.0.” https://docs.rs/sqlx/latest/sqlx/fn.query.html. Достъп: 4 ноември 2026.

[45] esqueleto contributors, “esqueleto-3.6.0.3: module `Database.Esqueleto.Experimental`.” https://hackage-content.haskell.org/package/esqueleto-3.6.0.3/docs/Database-Esqueleto-Experimental.html. Документация в Hackage. Достъп: 4 ноември 2026.

[46] Ellis, Tom and contributors, “opaleye-0.10.3.1: module `Opaleye.Table`.” https://hackage-content.haskell.org/package/opaleye-0.10.3.1/docs/Opaleye-Table.html. Документация в Hackage. Достъп: 4 ноември 2026.

[47] Ellis, Tom and contributors, “opaleye-0.10.3.1: an SQL-generating DSL targeting PostgreSQL.” https://hackage-content.haskell.org/package/opaleye-0.10.3.1/docs/Opaleye.html. Документация в Hackage. Достъп: 4 ноември 2026.

[48] Bock, Roland and contributors, “sqlpp11: a type safe embedded domain specific language for SQL queries and results in C++.” https://github.com/rbock/sqlpp11. Достъп: 4 ноември 2026.

[49] Bock, Roland and contributors, “sqlpp11 documentation: Functions, section on verbatim.” https://github.com/rbock/sqlpp11/blob/main/docs/Functions.md. Достъп: 4 ноември 2026.

[50] Bock, Roland and contributors, “sqlpp23 documentation: custom statements (verbatim and verbatim_table).” https://github.com/rbock/sqlpp23/blob/main/docs/custom_statements.md. Достъп: 4 ноември 2026.

[51] Bock, Roland and contributors, “sqlpp23: a type safe SQL library for C++.” https://github.com/rbock/sqlpp23. Достъп: 4 ноември 2026.

[52] Diesel project, “Function `diesel::debug_query`.” https://docs.diesel.rs/master/diesel/query_builder/fn.debug_query.html. Документация на разклонението master. Достъп: 4 ноември 2026.

[53] Bock, Roland and contributors, “sqlpp11 documentation: Prepared Statements.” https://github.com/rbock/sqlpp11/blob/main/docs/Prepared-Statements.md. Достъп: 4 ноември 2026.

ПРИЛОЖЕНИЯ

Приложение А. Възпроизвеждане на резултатите

Изходният код е публично достъпен в https://github.com/cpp-for-everything/ORM. Резултатите в глава VII са получени с версия `a17f4e7`. Наборът за оценка се изгражда и изпълнява със следните команди, от корена на хранилището:

```
cmake -S . -B build-security -DORM_ENABLE_SECURITY=ON -DORM_ENABLE_SQLITE=ON
cmake --build build-security
ctest --test-dir build-security -L security --output-on-failure
```

Listing 4: Изграждане и изпълнение на набора за оценка

Компилационните тестове изискват GCC или Clang. За изпълнение с Clang променливите на средата `CC` и `CXX` се задават съответно на `clang-18` и `clang++-18` преди конфигурирането. Резултатите се записват в `security/evidence/`. Описание на набора има в `security/README.md`.

Програмите за сървърните хранилища се изграждат, когато съответният конектор е включен, например с `-DORM_ENABLE_POSTGRESQL=ON`, и изискват работещ сървър, достъпен през променливите `ORM_*` на интеграционните тестове. Най-лесният начин е образът `test-all` с услугите от `docker/docker-compose.yml`, както го изпълнява работният процес `Docker Integration Tests`. Той качва каталога `security/evidence/` като артефакт `security-evidence`.

Приложение Б. Файлове на набора за оценка

Таблица 15. Съдържание на каталога `security`

Път	Съдържание
`compile_fail/`	18 компилационни теста и общата им постановка
`cmake/check_compile.cmake`	проверка на изхода и на диагностиката на всеки компилационен тест
`corpus/payloads.csv`	66 полезни товара с колони за идентификатор, клас, диалект, техника, текст, очакван ефект и източник
`corpus/sources.bib`	11 източника на корпуса
`runtime/differential_sqlite.cpp`	диференциалният тест със SQLite
`runtime/differential_postgresql.cpp`	диференциалните тестове със сървърите на PostgreSQL, MySQL, MongoDB, Redis и Cassandra и четири съседни
`runtime/differential_common.hpp`	общият драйвер на сървърните тестове: контрол, параметрирано заявяване, резюме и правило за неуспех
`runtime/test_injection_sqlite.cpp`	четири именувани теста на пътя през заместител
`probes/`	двете програми, възпроизвеждащи пропуска преди поправката
`evidence/`	всички записани резултати

Приложение В. Терминологичен речник

Таблица 16 съпоставя термините, използвани в текста, с английските им съответствия и с идентификаторите в изходния код.

Таблица 16. Терминологичен речник

Термин	На английски	В кода
инжекция	injection	
полезен товар	payload	колона `payload` в корпуса
екраниране	escaping	`to_sql_literal`
свързване на параметри	parameter binding	`sqlite3_bind_*`, `PQexecParams`
типизиран заместител	typed placeholder	`Placeholder<T>`, `ph<T, _N>`
литерален операнд	literal operand	`is_literal_operand_v`
междинно представяне	intermediate representation	`select_query` и сродните
свойство	property	`property<T, "name">`
конектор	connector	`connector_trait<DB>`
проверка на границата	boundary check	`check_query_shape`
контролен случай	control case	случаи 00, 14 и 15
оракул	oracle	функцията `classify`
затворен отказ	fail closed	
объркване на параметрите	parameter confusion	